EVALUATION-TIME CONSTANCY INFERENCE FOR TRANSPORT IN CUBICAL TYPE THEORY

RYOTA SHIBUSAWA

Daiichi Institute of Technology, Japan
e-mail address: r-shibusawa@daiichi-koudai.ac.jp

Abstract. The eliminator J of cubical type theories does not compute definitionally on refl: the transport primitive's constancy formula is fixed when a term is written, so constancy arising later by substitution is never used. Re-detecting it at evaluation time is an old informal idea; this paper gives it a metatheory. First, a no-go theorem: combining the evaluation-time rule equationally with the structural transport rules derives a constant-system hcomp-regularity principle — closely related to regularity principles known to fail in standard cubical-sets models — already at path types, before Glue; and restricting the structural rules to non-constant families is not substitution-stable. Second, the positive system: judgmental equality is defined as the algorithmic equality of a prioritized normalization-by-evaluation strategy whose constancy check is specified representation-independently (the fresh dimension must not occur in the read-back of the family's value) and implemented exactly; we prove the check semantically sound and the typed algorithmic equality an admissible conversion relation — equivalence, congruence, weakening, substitution, context conversion, type preservation — with termination and canonicity relative to an explicitly delimited base component, which we discharge for a Glue-free, universe-free core fragment: there, including for $\mathsf{J}\,d\,\mathsf{refl} \equiv d$ itself, the results are unconditional. All concrete positive and negative object-language conversion witnesses, including the switchover experiments behind the no-go theorem, are machine-checked in a self-contained Lean 4 kernel; the metatheory itself is pen-and-paper.

1. Introduction

Cubical type theories [1, 2] compute: univalence and higher inductive types have computational content, and canonicity and normalization hold [4, 5]. The price is paid at the eliminator J of Martin-Löf's identity type, which in cubical systems is a defined operation — transport along a connection square — and no longer satisfies the definitional computation rule $\mathsf{J}\,d\,\mathsf{refl} \equiv d$. In Cubical Agda [3] the equation holds only propositionally ($\mathsf{transportRefl}$); library code is peppered with explicit appeals to it, and the gap is a standing source of friction when porting Book-HoTT developments [9].

The mechanism is easy to isolate. The transport primitive $\mathsf{transp}^i\,A\,\varphi\,u$ of ABCFHL-style theories carries a *constancy formula* φ: a cofibration under which the family A is required to be judgmentally constant in i, with the primitive rule $\mathsf{transp}^i A\,\mathsf{i1}\,u \equiv u$. The formula is fixed when the term is *written*. Inside the definition of J the transport is formed

Key words and phrases: cubical type theory, definitional equality, regularity, transport, normalization by evaluation, De Morgan algebra.

Draft v12, 2026-07-17.

Preprint submitted to
Logical Methods in Computer Science

with $\varphi = \mathsf{i0}$; substituting $p := \mathsf{refl}$ later makes the family judgmentally constant, but nothing revisits φ. The information is available at evaluation time and is simply not used.

The obvious repair has circulated informally for years: let the *evaluator* re-detect constancy — normalize the family, check that the transport dimension does not occur, and if so return the input. Arend's coe computes on constant families over its (non-cubical) interval [8]. For full cubical theories the folklore assessment — which has, to our knowledge, circulated only informally — locates the obstruction in the interaction with the structural transport rules, above all at Glue, where regularity fails in the standard models [6, 7]; no published metatheoretic analysis of the idea for a univalent cubical calculus appears to exist. This paper provides that analysis: a no-go theorem locating the obstruction precisely, and the metatheory of an operational formulation that avoids it.

The no-go half. Write (R) for the evaluation-time rule: $\mathsf{transp}^i A\,\varphi\,u \equiv u$ whenever A is judgmentally i-constant, regardless of φ. When the family is constant, (R) and the *structural* rule for the family's head former both apply, and an equational theory containing both closes over the pair. We machine-check the resulting *switchover* equations former by former (Section 4). For Π, Σ and the argument-wise higher inductive constructors the structural contractum collapses back to the input — using, and in fact *forcing*, the η laws. For path types it does not: the structural rule

$$\mathsf{transp}^i(\mathsf{Path}\,A\,a\,b)\,\varphi\,p \;\equiv\; \langle j\rangle\,\mathsf{comp}^i\,A\,[\,j{=}0 \mapsto a,\;\; j{=}1 \mapsto b\,]\,(p\,j)$$

leaves, at a constant family, an hcomp whose tubes are constant in the fill dimension. Equating that residue with p is a constant-system homogeneous-composition regularity principle — an equation of the kind excluded in the standard cubical-sets models [6, 7], and one our kernel refutes ($\mathsf{trans}\,\mathsf{refl}\,\mathsf{refl} \not\equiv_{\mathrm{alg}} \mathsf{refl}$). Thus (Theorem 4.6, with the full derivation in Section 4.3): *any equational theory containing the structural path-transport rule, rule (R), congruence and transitivity derives constant-tube* hcomp*-regularity; the conflict arises at path types already — before, and independently of,* Glue. Moreover the alternative of *restricting* the structural rules to non-constant families — the route taken by an earlier version of this work — fails for a dual reason: non-constancy is *not* stable under substitution (a family non-constant at definition time is exactly one that may become constant later), so the restricted rule set is not closed under substitution either.

The positive half. The way out is to stop generating the equality by rules on terms and instead take the judgmental equality of the extended system to *be* the algorithmic equality of a prioritized evaluation strategy: normalization by evaluation, in which a transport node first evaluates its family, instantiates it at a fresh dimension, and checks constancy *of the resulting value*; only if the check fails does it dispatch structurally. For this to be a definition of judgmental equality rather than a description of an implementation, a body of metatheory is owed, and this paper's main technical contribution is to supply it (Sections 5–9):

- a *representation equivalence* $\approx$ on the semantic domain (values differing only in closure representation), because in a defunctionalized NbE the two sides of the substitution lemma do *not* produce identical values, only $\approx$-related ones (Section 5);
- a *specification* of the constancy check — the fresh dimension must not occur in the *read-back* of the family value — which is representation-independent, together with its *semantic soundness theorem*: a checked family has logically-related endpoints (Section 6); the implemented occurs-check is proved to *decide the specification exactly* (Theorem 6.3), so the kernel and the specified system are one system, not two;

- the substitution lemma $\mathsf{eval}(t\sigma, \rho) \approx \mathsf{eval}(t, \mathsf{eval}(\sigma, \rho))$, whose transport case goes through *because* the check is specified representation-independently — the branch taken is a function of the $\approx$-class (Section 7);
- a Kripke logical relation $\equiv^{\mathrm{val}}$ over the domain whose fundamental theorem yields, as corollaries, that the typed algorithmic equality is an *admissible conversion relation*: an equivalence, a congruence for every term former, stable under weakening, substitution and context conversion, type-preserving, and coherent with the bidirectional checker (Theorem 7.8);
- termination folded into the logical relation in the standard Tait style — no syntactic termination measure is claimed, and the double induction (typing derivation outside, semantic type values inside) discharging the structural transport branch is made explicit (Section 9);
- canonicity: closed naturals evaluate to numerals, whence consistency and $\mathsf{J}\, d\, \mathsf{refl} \equiv d$ (Theorem 9.1). The base-rule cases of the fundamental theorem are inherited as an explicitly delimited *base component* (Section 2.2);
- an *unconditional core*: for the minimal fragment $\Pi, \Sigma, \mathsf{Path}, \mathbb{N}$ with transport and homogeneous composition — no Glue, no universes, no HITs — we discharge the base component ourselves, in full per-former detail (Appendix F), so that there admissibility, canonicity and the definitional $\mathsf{J}\, d\, \mathsf{refl}$ hold with no assumption at all (Theorem 9.2); the derived J and its computation on refl are spelled out as an explicit proposition (Proposition F.29). The extension of the core by S^1 is stated separately as a proposition with a proof sketch and artifact evidence, so the headline result is not entangled with higher-inductive obligations;

The three layers, once and for all. The paper's positive results live at three clearly separated strengths, and we keep the labels attached throughout:

- *Unconditional theorem* (Theorem 9.2, Appendix F): the Glue-free, universe-free *minimal core* $\Pi, \Sigma, \mathsf{Path}, \mathbb{N}$ with transp and hcomp — large enough to define J, so $\mathsf{J}\, d\, \mathsf{refl} \equiv d$ is unconditional;
- *Relative theorem* (Theorems 7.6, 7.8, 9.1): the full implemented De Morgan calculus, relative to the delimited base component (B) of Section 2.2;
- *Artifact evidence*: Glue, univalence, and the HIT roster, machine-checked in the kernel but not covered by the unconditional layer.

When we speak of a metatheory "for a univalent cubical calculus", univalence lives in the second and third layers; the unconditional layer is deliberately Glue- and universe-free.

We claim neither full regularity (the catalogue records $\mathsf{trans}\, p\, \mathsf{refl} \not\equiv_{\mathrm{alg}} p$: hcomp is untouched, as by Theorem 4.6 it must be), nor a semantic countermodel for the no-go equation (we state precisely what is refuted algorithmically and what is conjectured semantically), nor a model construction for the positive system; open problems are collected in Section 13. A complementary result — on a generic path, algorithmic equality of connection-reparametrizations is exactly free De Morgan equality — is kept in Appendix B.

2. The base calculus

2.1. **Syntax and rules.** We fix an ABCFHL-style univalent cubical type theory T_0 with a De Morgan interval.[1] The judgment forms, formers and rules used in this paper:

- *Interval and cofibrations.* Interval expressions over $i_1, \dots, i_n$ form the free De Morgan algebra $\mathrm{DM}(i_1, \dots, i_n)$; cofibrations φ are generated by face equations $i{=}0$, $i{=}1$, conjunction and disjunction, with constants $\mathsf{i0}, \mathsf{i1}$.
- *Formers.* Π, Σ, Path (heterogeneous PathP), universes à la Russell, natural numbers and base types, Glue, and the higher inductive types listed in Section 4.2.
- *η rules* for Π, Σ and Path.
- *Transport*, with an explicit constancy formula:

$$\frac{\Gamma, i : \mathbb{I} \vdash A\ \mathsf{type} \qquad \Gamma \vdash \varphi\ \mathsf{cof} \qquad \Gamma, \varphi, i : \mathbb{I} \vdash A \equiv A(\mathsf{i0}/i) \qquad \Gamma \vdash u : A(\mathsf{i0}/i)}{\Gamma \vdash \mathsf{transp}^i A\, \varphi\, u : A(\mathsf{i1}/i)}$$

  with the primitive collapse rule $(\mathrm{S}_{\mathsf{i1}})$: $\mathsf{transp}^i A\, \mathsf{i1}\, u \equiv u$ and one *structural* rule per former; the three needed later (φ omitted where it merely distributes):

$$\mathsf{transp}^i(\Pi x{:}A.B)\, u \equiv \lambda x_1.\, \mathsf{transp}^i B[w\, x_1\, i/x]\, \big(u\, (w\, x_1\, 0)\big), \qquad w\, x_1\, i := \mathsf{transp}^j A(i \vee \neg j)\, x_1;$$

$$\mathsf{transp}^i(\Sigma x{:}A.B)\, u \equiv \big(\mathsf{transp}^i A\, (\mathsf{fst} u),\ \mathsf{transp}^i B[\overline{u}\, i/x]\, (\mathsf{snd} u)\big), \quad \overline{u}\, i := \mathsf{transp}^j A(i \wedge j)\, (\mathsf{fst} u);$$

$$\mathsf{transp}^i(\mathsf{Path}\, A\, a\, b)\, p \equiv \langle j \rangle\, \mathsf{comp}^i A\, [\, j{=}0 \mapsto a,\ j{=}1 \mapsto b\,]\, (p\, j).$$

  Higher inductive types transport argument-wise on constructor arguments (Section 4.2); Glue transports by the composite operation $\mathsf{transpGlue}$ of [1, §6.2], recalled in Section 4.4.
- *Homogeneous composition* hcomp with the usual boundary rules. *No* regularity rule for hcomp is assumed, and none may be: constant-tube collapse is of the kind excluded in the standard models [6, 7]. comp abbreviates transport followed by hcomp of the transported tubes.

2.2. **The base component, and the three-layer main theorem.** The metatheory below is organized so that everything specific to rule (R) is proved here, while the (large) base-theory content enters as one delimited component:

(B) *The base component*: the Kripke logical relation of Section 5 restricted to T_0, i.e. the predicate clauses for all T_0 formers, the read-back correctness lemmas ($v \equiv^{\mathrm{val}} \mathsf{eval}(\mathsf{quote}\, v)$, uniqueness and inconvertibility of distinct normal forms, injectivity of type constructors), and the fundamental-theorem cases for all T_0 rules, in environment form.

Our results then come in three layers:

- **Theorem A′ (unconditional minimal core).** For the fragment $\Pi, \Sigma, \mathsf{Path}, \mathbb{N}$ with transp and hcomp — no Glue, no universes, no HITs — component (B) is discharged *in this paper* (Appendix F), in per-former detail: because the fragment has no universe, its logical relation is defined by *structural recursion on syntactic types in environments* — a plainly well-founded induction — and because it has no Glue, transport never meets the $\mathsf{transpGlue}$ machinery. All positive results are unconditional there (Theorem 9.2); in particular $\mathsf{J}\, d\, \mathsf{refl} \equiv d$, whose definition needs only paths, connections and transport, is

[1] The Cartesian variant carries the switchover analysis through unchanged — the residues are about hcomp, not connections — and strengthens the metatheory (Theorem A below). We fix the De Morgan case as primary because the accompanying implementation realizes it.

unconditional (Proposition F.29). The S^1 extension is a separate proposition (Appendix F, last subsection).

- **Theorem A (Cartesian layer).** For Cartesian cubical type theory, the judgmental-theory-level part of component (B) — the normal-form language and bijection, inconvertibility of distinct normal forms, injectivity of type constructors, decidability — is discharged by the normalization theorem of Sterling–Angiuli [5] (normal-form specification in [10]), *for the fragment their theorem covers*: Π, Σ, path, glue and the circle, *without universes*. The evaluator-level part of (B) — the concrete Kripke relation over our domain and the environment-form fundamental cases — is not literally in [5], whose method is deliberately not evaluation-based; it is a standard NbE-correctness transposition, which we enumerate but do not claim from the literature. Appendix G gives the precise item-by-item correspondence, the translation between their boundary-conditioned coe and our φ-annotated transp, and the exact residual gaps (universes; the wider HIT roster; the evaluator-level transposition). Theorem A is therefore unconditional only over that fragment and modulo that transposition; we do not claim more.
- **Theorem B (De Morgan layer).** For the De Morgan calculus — which the accompanying implementation realizes — component (B) is an assumption: Huber [4] provides its closed/canonicity fragment, but a full open-term normalization proof on the De Morgan side is not in the literature. All theorems below are stated relative to (B).

3. Rule (R), and the instability of non-constancy

Definition 3.1 (Judgmental constancy)**.** A is *judgmentally i-constant* (over $\Gamma, i : \mathbb{I}$) if there is A_0 with $\Gamma, i \vdash A \equiv A_0$ and $i \notin \mathsf{FV}(A_0)$.

The rule under study is

$$\text{(R)} \qquad \mathsf{transp}^i A\, \varphi\, u \equiv u \qquad \text{for } \textit{any } \varphi, \text{ whenever } A \text{ is judgmentally } i\text{-constant.}$$

Write $T_{\mathrm{eq}} := T_0 + (\mathrm{R})$ for the *unrestricted equational theory*. Two different systems appear in this paper, and we name them apart once and for all: T_{eq} is the object of the no-go analysis (Section 4) and the target of the soundness theorem (Section 8); it is not used as the type system of record (its judgmental equality appears in statements, its typing never does). The system of record — in which every fundamental-theorem, admissibility and canonicity statement below is made — is the type system T_{alg} of Definition 7.2, whose conversion rule uses the algorithmic equality.

Remark 3.2 (Why the explicit formula matters)**.** In a φ-free presentation whose collapse rule is the side condition "$i \notin \mathsf{FV}(A) \Rightarrow \mathsf{transp}^i A\, u \equiv u$", rule (R) is derivable from congruence. With the explicit formula — as in ABCFHL and Cubical Agda — it is not: (S_{i1}) applies only at $\varphi = \mathsf{i1}$, and congruence cannot change φ. So (R) genuinely extends the judgmental equality of the φ-explicit calculus; for the φ-free reading, the contribution of this paper is the operational system of Section 7.

Lemma 3.3 (Positive substitution stability)**.** *If A is judgmentally i-constant over Γ, i and $\sigma : \Delta \to \Gamma$, then $A\sigma$ is judgmentally i-constant over Δ, i.*

Proof. Judgmental equality is closed under substitution; i is bound at the transport site, may be chosen fresh for Δ, and the terms of σ live over Δ, so $\mathsf{FV}(A_0\sigma) \not\ni i$. □

Lemma 3.4 (Type preservation)**.** *If A is judgmentally i-constant then $A(\mathsf{i0}) \equiv A_0 \equiv A(\mathsf{i1})$; rule (R) preserves typing.*

Note the asymmetry: constancy is substitution-stable; *non*-constancy is not — substituting refl for a path variable is exactly how a non-constant family becomes constant. Any side condition of the form "the family is *not* constant" is therefore unusable in a rule schema, both because it is not preserved by substitution and because, as a negative premise, it does not even define an inductive rule set.

4. The switchover analysis and the no-go theorem

When the family is constant, (R) and the structural rule for its head former both apply. Call the induced equation between the two contracta the *switchover equation* of the former. This section determines, former by former and machine-checked throughout, which switchover equations hold in the algorithmic equality (Section 7); the probes are the file `LibSwitchover.lean` of the artifact.

4.1. Convergent formers.

Lemma 4.1 (Switchover convergence at Π, Σ)**.** *Let the families involved be judgmentally i-constant. Then, in $T_0 + (\mathrm{R})$ and in the algorithmic equality:*

$$\lambda x.\, \mathsf{transp}^i B\,\big(u\,(\mathsf{transp}^i A\, x)\big) \equiv u \qquad (\Pi)$$

$$\big(\mathsf{transp}^i A\,(\mathsf{fst} u),\ \mathsf{transp}^i B'\,(\mathsf{snd} u)\big) \equiv u \qquad (\Sigma,\ \textit{incl. dependent})$$

Proof. Π: the inner families are reparametrizations of constant families, hence constant; (R) collapses both inner transports, leaving $\lambda x.\, u\, x$, and η_Π finishes. Σ: in the dependent case the second family is $B[\overline{u}\, i/x]$ with $\overline{u}\, i \equiv \mathsf{fst} u$ by (R), hence constant; (R) twice, then η_Σ. Machine witnesses: `swPiD`, `swSigD`, `swSigDepD` (all accepted). □

Remark 4.2 ((R) forces η)**.** Over an η-free base, the Σ switchover equation *is* surjective pairing: any theory containing (R) and structural Σ-transport derives η_Σ (similarly η_Π). Our kernel has all three η laws, machine-checked, as any implementation of (R) must.

4.2. Higher inductive types: the argument-wise class.

For higher inductive types we make no blanket claim; we delimit a signature class and check it.

Definition 4.3 (Argument-wise transport signatures)**.** A constructor signature is *argument-wise* if its structural transport transports each point argument along the corresponding line and carries interval arguments unchanged, generating no boundary correction (hcomp/filler).

Lemma 4.4. *For argument-wise signatures, the switchover equation converges by (R) on each argument line alone (no η needed): the contractum is literally the constructor form.*

The signatures implemented in the artifact classify as follows (machine witnesses in the last column):

HIT	constructors	class	switchover
lists	nil, cons	argument-wise	converges (`swListD`)
suspension	N, S, merid	arg.-wise (path ctor.)	converges (`swSuspMeridD`)
pushout	inl, inr, push	argument-wise	converges (same schema)
S^1, torus	point/loop/surface	no point arguments	trivially converges
$K(G,1)$, BGpd	base/loop/comp cells	argument-wise	converges (same schema)
quotient	qin	argument-wise	converges
quotient	qeq, squash cells	*no structural rule*	vacuous (no critical pair)
truncation	tin, squash	argument-wise / none	converges / vacuous

In our kernel, transports of non-constructor elements of quotients and truncations become neutral, so for the squash-type constructors there is no structural contractum and hence no switchover question. Signatures whose structural transport would generate boundary corrections (constructors with non-trivial boundary in the transported arguments) are *not* covered by Lemma 4.4; in the light of Section 4.3 we expect non-convergence there, and we leave a general signature framework as future work.

4.3. **Non-convergence at path types.** At a constant family $\mathsf{Path}\,A\,a\,b$, the transport component of the structural contractum collapses by (R), and the residue is an hcomp whose tubes are constant in the fill dimension.

Lemma 4.5 (Path separation)**.** *In the algorithmic equality, over the generic context* $(A{:}\mathcal{U},\ a\,b{:}A,\ p{:}\mathsf{Path}\,A\,a\,b)$*,*

$$\langle j\rangle\,\mathsf{hcomp}^{\iota}\,A\,[\,j{=}0\mapsto a,\ j{=}1\mapsto b\,]\,(p\,j)\ \not\equiv_{\mathrm{alg}}\ p.$$

(Negative probe in `LibSwitchover.lean`*; a control probe confirms the left-hand side is well-typed, so the failure is one of conversion, not typing.) For the* minimal core *the schema version — A any fixed core type — is moreover proved on paper, unconditionally, in Proposition F.28; the machine probe remains the witness for the full implemented calculus, where A is a universe variable.*

The derivation of that equation in any equational extension is short and fully formal. Write $P := \mathsf{Path}\,A\,a\,b$ (so $i \notin \mathsf{FV}(P)$) and fix the typing premises $\Gamma \vdash A : \mathcal{U}$, $\Gamma \vdash a, b : A$, $\Gamma \vdash p : P$; note $\Gamma, i \vdash P\ \mathsf{type}$ and $\Gamma \vdash \mathsf{transp}^{i} P\,\mathsf{i0}\,p : P$.

$$\Gamma \vdash \mathsf{transp}^{i} P\,\mathsf{i0}\,p \equiv p \qquad \text{by (R), } P \text{ } i\text{-free} \tag{4.1}$$

$$\Gamma \vdash \mathsf{transp}^{i} P\,\mathsf{i0}\,p \equiv \langle j\rangle\,\mathsf{comp}^{i} A\,[\partial j \mapsto a, b]\,(p\,j) \qquad \text{structural } \mathsf{Path} \text{ rule} \tag{4.2}$$

$$\Gamma \vdash \langle j\rangle\,\mathsf{comp}^{i} A\,[\cdots]\,(p\,j) \equiv \langle j\rangle\,\mathsf{hcomp}^{\iota} A\,[\cdots]\,(p\,j) \qquad \mathsf{comp} = \mathsf{hcomp} \circ \mathsf{transp};\ \text{(R)};\ \text{congr.} \tag{4.3}$$

$$\Gamma \vdash p \equiv \langle j\rangle\,\mathsf{hcomp}^{\iota} A\,[\,j{=}0\mapsto a,\ j{=}1\mapsto b\,]\,(p\,j) \qquad \text{sym., trans. of (4.1)–(4.3)} \tag{$\dagger$}$$

Each step preserves the stated typing: (4.1) by Lemma 3.4; (4.2) is the structural rule at its stated type; in (4.3) the transported tubes and base are along i-free lines, so (R) applies under the binder and the congruence rules for hcomp systems, and the boundary conditions ($j{=}0$: base $p\,0 \equiv a$ matches the tube; $j{=}1$ symmetrically) are exactly those of the structural rule.

Four remarks on scope, answering natural objections. (i) The rule used in (4.2) is the Path-instance of the transport unfolding common to CCHM [1, §3] and ABCFHL [2]; the heterogeneous PathP instance differs only in that the composition line is $A(i, j)$, and at a constant family degenerates to the same residue. (ii) The residue survives in the

Cartesian calculus unchanged — no connections are used anywhere in (4.1)–(†). (iii) η_{Path} cannot discharge (†): η cancels a path abstraction against a path application, and the right-hand side's body is an hcomp value, not an application of p. (iv) "Constant tubes" means constancy in the *fill* dimension ι; the tubes do depend on j.

Theorem 4.6 (No-go)**.** (1) *Any equational theory containing the structural path-transport rule, rule (R), congruence and transitivity derives* (†)*. Consequently the conversion checker of Section 7, which refutes* (†) *(Lemma 4.5), is incomplete for every such theory.*
(2) *Equation* (†) *is a constant-system* hcomp*-regularity principle. We do not claim it is literally an instance of a published counterexample; it is closely related to the regularity principles excluded by the results of Swan* [6] *and Sattler* [7]*, and we conjecture that it fails in the standard cubical-sets semantics. Constructing a countermodel is stated as an open problem (Section 13); no result of this paper depends on it.*
(3) *Restricting the structural transport rules to judgmentally non-constant families does not define a substitution-closed rule set (Section 3); closing the restricted theory under substitution re-imports* (†)*. An explicit witness: over* $\Gamma = (A : \mathcal{U},\ a\, b : A,\ p : \mathsf{Path}\, A\, a\, b)$ *the family* $C(i) := \mathsf{Path}\, A\, a\, (p\, i)$ *is judgmentally* non*-constant (its normal form mentions* i *through the neutral* p*), so the restricted theory applies the structural rule:* $\mathsf{transp}^i C\, \mathsf{i0}\, q \equiv \langle j \rangle\, \mathsf{comp}^i A\, [\ldots]\, (q\, j)$*. Substituting* $\sigma := [\mathsf{refl}_a/p]$ *(and* a *for* b*) sends* C *to the* constant *family* $\mathsf{Path}\, A\, a\, a$*; the substituted instance of the derived equation must still hold in a substitution-closed theory, while (R) now collapses its left-hand side — transitivity yields exactly* (†) *at* $\mathsf{Path}\, A\, a\, a$*.*
(4) *The conflict is independent of* Glue*: the derivation uses path types only.*

Theorem 4.6 is, we believe, the precise content of the folklore judgment that evaluation-time constancy detection "does not interact well with the rest of cubical type theory": the obstruction is not the detection, and not specifically Glue, but that any equational marriage of (R) with the hcomp-generating structural rules collapses onto regularity for hcomp. It also fixes the design space: the extension must be operational, with (R) tried *before* structural dispatch.

4.4. **The** Glue **residue.** The same computation for Glue shows its switchover residue is the path residue plus fiber corrections. Let $G := \mathsf{Glue}\, [\varphi \mapsto (T, w)]\, B$ with φ, T, w, B all i-free, and $u_0 : G$. Unfold transpGlue [1, §6.2] at the constant line, collapsing every inner transport by (R): the branch transport $\mathsf{transp}^i T\, u_0$ collapses; the branch filler line becomes the *constant* tube $\iota \mapsto w.1\, u_0$; hence the corrected base is

$$a_1' \;=\; \mathsf{hcomp}^{\iota} B\, [\, \delta \mapsto w.1\, u_0\,]\, (\mathsf{unglue}\, u_0), \qquad \delta = \forall i.\, \varphi = \varphi,$$

a constant-tube hcomp, stuck off δ — the residue of Lemma 4.5 — after which the fiber-point correction adds further hcomps on the fiber Σ-type before reassembly with glue.

5. The semantic domain, and two value equivalences

This section and the next two supply the metatheory that entitles us to call an algorithmic equality a *judgmental* equality. We work with the concrete NbE domain of the implementation; all definitions are stated so as to be implementation-independent.

5.1. **The domain.** *Levels* $\ell \in \mathbb{N}$ name free variables (term and dimension alike); a *depth* d records the number of enclosing binders, with the invariant that every free level of a value produced at depth d is $< d$. *Interval values* are elements of the free De Morgan algebra over levels, with a canonical antichain-DNF. *Values* are weak-head normal: a head constructor with value, interval, cofibration, system and *closure* arguments. *Closures* are defunctionalized: either $\mathsf{mk}(\rho, t)$ — an environment (a list of values) and a term body — or one of finitely many closure combinators (reparametrizations, the structural-transport auxiliaries, ...); *instantiation* $\mathsf{inst}_d(c, w)$ dispatches on the combinator and, in the mk case, calls $\mathsf{eval}(t, \rho{+}[w])$ — the one place evaluation re-enters, and the reason no syntactic termination measure exists. *Neutrals* are elimination spines on a variable level. The *dimension action* $v\langle\xi\rangle$ of a partial map $\xi : \mathrm{Level} \rightharpoonup \mathbb{I}$ applies ξ to interval parts, preserving heads and acting pointwise on closure environments.

5.2. **Representation equivalence.** In a defunctionalized NbE, the values $\mathsf{eval}(A\sigma, \rho)$ and $\mathsf{eval}(A, \mathsf{eval}(\sigma, \rho))$ are *not* identical: the first carries the substituted body with a shorter environment, the second the original body with a longer one. Every "equal values" statement below is therefore made with respect to:

Definition 5.1 (Representation equivalence $\approx$)**.** The largest relation such that $v \approx v'$ implies: same head constructor; interval and cofibration arguments *literally equal* (in DNF); value arguments $\approx$; neutral spines pointwise (same head level, arguments $\approx$); and for closures, $c \approx c'$ iff $\mathsf{inst}_d(c, w) \approx \mathsf{inst}_d(c', w)$ for all d and all arguments w.

Literal equality of interval parts is meaningful because fresh levels are generated deterministically by depth, and the two evaluations traverse the same term structure at the same depths (the depth invariant of Section 5.1; this is made a standing lemma in Appendix C).

Lemma 5.2 (Read-back invariance)**.** $v \approx v' \implies \mathsf{quote}_d(v) = \mathsf{quote}_d(v')$.

Proof. quote dispatches on heads, copies interval parts, and opens closures at generic levels — exactly the observations $\approx$ preserves; coinduction over the read-back. □

$\approx$ is preserved by all semantic operations (inst, application, projections, path application, transp, hcomp, $\mathsf{glue}/\mathsf{unglue}$): each dispatches on heads, interval parts and instantiation behavior only. The per-operation compatibility lemmas are stated in Appendix C.

5.3. **The Kripke logical relation.**

Definition 5.3 ($\equiv^{\mathrm{val}}$)**.** By induction on type values (stratified by universe level), a Kripke partial equivalence relation. In this section, for readability, worlds are depths; the full world structure — depths together with cofibration assumptions, with dimension substitutions and face restrictions as maps — is what the clauses quantify over wherever systems and faces appear, and is made fully explicit for the core in Appendix F.2 (for the general calculus it is part of component (B), as in [4], whose computability predicates are likewise face-indexed families):

- $\equiv^{\mathrm{val}}_{\mathbb{N}}$: forced to the same numeral, or both neutral with equal read-back;
- $f \equiv^{\mathrm{val}}_{\Pi(D,C)} f'$ iff for all $d' \geq d$ and all $w \equiv^{\mathrm{val}}_{D} w'$: $f\, w \equiv^{\mathrm{val}}_{\mathsf{inst}(C,w)} f'\, w'$;
- Σ: componentwise; $\mathsf{PathP}(F, l, r)$: at every interval value, in the corresponding fiber;

- Glue: the unglued parts related in the base, and on each live face the branch parts related in the branch type;
- $\mathcal{U}$: both codes decode to related PERs (the universe induction);
- HITs: constructor-wise (value arguments related, interval arguments literally equal), closed under hcomp and neutrals.

Lemma 5.4. *(i)* $\approx \subseteq \equiv^{\mathrm{val}}_T$ *for every* T*. (ii)* $\equiv^{\mathrm{val}}$ *is a PER, invariant under* $\equiv^{\mathrm{val}}$*-replacement of the type value. (iii)* Conversion correctness*:* $\mathsf{conv}_d(v, v', T)$ *accepts iff* $v \equiv^{\mathrm{val}}_T v'$*. Soundness of* conv *(accepted implies related) is by induction on the run: each clause checks exactly a defining clause of* $\equiv^{\mathrm{val}}$*. Completeness (related implies accepted) is the standard NbE read-back argument, part of the base component (B) for base types and extended in Appendix C for the new clause. In particular symmetry and transitivity of the algorithmic equality are inherited from the PER structure.*

Remark 5.5 (Where clause (iii) is proved, per layer)**.** In the *relative* full calculus, clause (iii) — in particular its term-level soundness direction — is part of the base component (B) of Section 2.2, inherited in the form stated. For the *unconditional core*, Appendix F does *not* re-prove term-level soundness and does not need it: it replaces clause (iii) by the read-back characterization of conversion (Lemma F.16), type-level soundness (Lemmas F.17–F.19) and completeness (Lemma F.20); see Remark F.21 for why this weaker package suffices for every use the unconditional results make. The two proof routes coexist without conflict, but they are different routes, and each theorem below cites the one appropriate to its layer.

6. The constancy check: specification, soundness, implementation

6.1. **Specification.** The system is *defined* with the following representation-independent check. Let F be the family value instantiated at a fresh dimension level ℓ at depth $d{+}1$.

Definition 6.1 (Check specification)**.** $\mathsf{const?}_{\mathrm{spec}}(F, \ell) :\iff \ell \notin \mathsf{FV}(\mathsf{quote}_{d+1}(F))$.

By Lemma 5.2 the specification is invariant under $\approx$ — the property that makes the substitution lemma below go through, and that no direct traversal of representations can have.

6.2. **Semantic soundness.**

Theorem 6.2 (Constancy-check soundness)**.** *If* $\mathsf{const?}_{\mathrm{spec}}(F, \ell)$ *then for all interval values* r, s*:* $F\langle r/\ell\rangle \equiv^{\mathrm{val}} F\langle s/\ell\rangle$ *(as type values: they decode to the same PER).*

Proof. (1) *Read-back/action commutation*: $\mathsf{quote}(v\langle\xi\rangle) = \mathsf{quote}(v)\langle\xi\rangle$ — the action touches only interval parts and closure environments, and read-back opens closures at generic levels disjoint from $\operatorname{dom}\xi$; induction over the read-back. (2) Hence with $\ell \notin \mathsf{FV}(\mathsf{quote}F)$: $\mathsf{quote}(F\langle r/\ell\rangle) = \mathsf{quote}(F)\langle r/\ell\rangle = \mathsf{quote}(F) = \mathsf{quote}(F\langle s/\ell\rangle)$. (3) *NbE idempotence at type values*: the family value and the evaluation of its read-back determine the same relation. For the general calculus this is part of the base component (B) (Appendix C); for the minimal core it is proved as an independent lemma (Lemma F.17), by structural induction on the family's syntactic type — read-back equality and quote–eval idempotence are distinct theorems, and we use the latter, at type values only. Then $F\langle r/\ell\rangle \equiv^{\mathrm{val}} \mathsf{eval}(\mathsf{quote}F) \equiv^{\mathrm{val}} F\langle s/\ell\rangle$. □

Theorem 6.2 is what entitles the positive branch of the evaluator to return its input: the two endpoint type values have *equal read-back* and decode to the same PER, so no coercion is needed — see the transport case of Theorem 7.6.

6.3. **The implementation realizes the specification exactly.** The kernel's occurs-check (`usesLvl`) mirrors the read-back traversal clause by clause: the same face forcing at the head, the same structural recursion on value and interval arguments, binder closures instantiated at generic levels exactly as read-back's binder cases, and the tubes of neutral hcomps likewise instantiated, exactly as read-back's system case. Its two optimizations are semantically transparent: an $O(1)$ *level-bound cut* — every value carries an upper bound on its free levels, maintained by the smart constructors; the bound's correctness (Lemma L1, Appendix C.1) makes the cut *exact* — and memoization.

Theorem 6.3 (Checker exactness, Kleene form)**.** *For every value F and level ℓ: if either of $\mathit{usesLvl}(\ell, F)$, $\mathsf{quote}(F)$ is defined then both are, and then*

$$\mathit{usesLvl}(\ell, F) = \mathsf{true} \iff \ell \in \mathsf{FV}(\mathsf{quote}(F)).$$

Hence the implemented check decides $\mathsf{const?}_{\mathrm{spec}}$ exactly on exactly the inputs where the specification is defined; in particular it is invariant under $\approx$ (by Lemma 5.2), although its traversal never constructs the read-back term.

Proof. Simultaneous induction on the defining big-step derivations (Section 7.1) of `usesLvl` and quote, whose recursion structures are mirrored clause by clause; the full clause table, covering every value, neutral and closure-instantiation case, together with the definedness transfer, the completeness of the level-bound cut, and the transparency of memoization, is Appendix E. □

Remark 6.4 (An instructive archaeology)**.** An earlier version of the checker walked the tubes of neutral hcomps *structurally*, over their closure environments, without instantiation — a representation-sensitive over-approximation. That design would have opened a real gap: a family the specification accepts but the implementation declines, making the specification take the (R)-branch and the implementation the structural branch — whose results, at path types, are *deliberately* non-convertible (Lemma 4.5); one-way soundness of the check would then not make the kernel a realization of the specified system at all. Inspection during the present revision showed that code path had become unreachable — the live path already instantiates, mirroring read-back — and it has been removed from the artifact, with the exact-mirroring property now asserted in the kernel's documentation. We record this because it sharpens Remark 7.4 into a slogan: *any deviation of the checker from read-back is not a harmless approximation but a change of judgmental equality.*

Consequently the implemented evaluator *is* the specified system: every branch decision agrees (Theorem 6.3), all theorems of Sections 7–9 apply to the implementation verbatim, and every machine probe — positive and negative — is a claim about the one algorithmic equality.

7. The operational system and its admissibility

7.1. **Partiality: what kind of objects the definitions are.** The functions eval, inst, quote, $\mathsf{const?}_{\mathrm{spec}}$ (which calls quote) and conv form *one* mutually recursive family of *partial* functions. Mathematically, we define their *graphs* inductively, as big-step derivations: a derivation certifies "this call is defined, with this result"; determinism is by construction of the rules, and the implementation's recursive definitions compute exactly these graphs. Two call cycles are inherent and intended: instantiation re-enters evaluation on stored closure bodies (Section 5.1), and the transport clause calls the check, which reads back the family value, which re-instantiates closures — $\mathsf{eval} \to \mathsf{const?}_{\mathrm{spec}} \to \mathsf{quote} \to \mathsf{inst} \to \mathsf{eval}$. Neither is vicious: inductive definitions of graphs need no termination argument; *totality on well-typed inputs* is a theorem (Corollary 7.7), proved by the logical relation and never assumed. Three consequences for the statements below: the closure clause of $\approx$ (Definition 5.1) is read Kleene-style — for all w, $\mathsf{inst}(c, w)$ and $\mathsf{inst}(c', w)$ are both undefined or both defined and related; the substitution lemma is stated in Kleene form; and the compatibility lemmas of Appendix C.2 are simultaneous inductions on the defining derivations, with coinduction on the produced values.

Formally, $\approx$ is the greatest fixed point of an operator Φ on relations over the partial computation graph: for a relation R on values, $\Phi(R)$ relates v, v' iff they have the same head constructor, literally equal interval and cofibration arguments, R-related value arguments, pointwise-R neutral spines, and, for each closure argument pair (c, c') and every w, either both $\mathsf{inst}(c, w)$ and $\mathsf{inst}(c', w)$ are undefined or both are defined and R-related. Φ is monotone (it uses R only positively), so $\approx := \nu\Phi$ exists by Knaster–Tarski; coinductive proofs below exhibit Φ-invariant relations (post-fixed points), and their guardedness is the fact that each appeal to the coinduction hypothesis sits under at least one value constructor of Φ's unfolding. The simultaneous inductions of Appendix C.2 are ordered by the sub-derivation relation of the big-step graphs — an evaluation derivation strictly contains the derivations of its premises, and instantiation derivations are premises of the evaluation derivations that invoke them — which is well-founded because derivations are finite trees; no measure on values or terms is used anywhere.

7.2. **The prioritized evaluator.** $\mathsf{eval}(t, \rho)$ is the base NbE evaluator with one changed clause. On $\mathsf{transp}^i\, A\, \varphi\, u$:

(1) $F := \mathsf{inst}_{d+1}(\mathsf{eval}(A, \rho), \ell)$ at a fresh dimension level ℓ;
(2) if $\mathsf{const?}_{\mathrm{spec}}(F, \ell)$ (implementation: `usesLvl`), return $\mathsf{eval}(u, \rho)$;
(3) otherwise dispatch structurally on the head of F as in the base evaluator (Π, Σ, Path, HITs, Glue via $\mathsf{transpGlue}$, neutral).

The check runs on the *evaluated* family, so it fires on families whose value is constant even when their syntax is not (machine-checked: `strictTranspValueConstD`). There is exactly one definition of constancy in the system — the one the evaluator runs.

Definition 7.1 (Typed algorithmic equality)**.** $\Gamma \vdash t \equiv_{\mathrm{alg}} u : A$ iff

$$\mathsf{conv}_{|\Gamma|}\big(\mathsf{eval}(t, \rho_\Gamma),\ \mathsf{eval}(u, \rho_\Gamma),\ \mathsf{eval}(A, \rho_\Gamma)\big)$$

accepts, where ρ_Γ is the generic environment of Γ (each variable a fresh neutral at its type).

Definition 7.2 (The system of record T_{alg})**.** T_{alg} is the type system with the rules of Appendix A whose conversion rule is mediated by $\equiv_{\mathrm{alg}}$ (Definition 7.1); it is the system the bidirectional checker implements. It is *not* presented by an equational theory on terms: by Theorem 4.6 no substitution-closed rule-generated presentation containing both (R) and the structural rules can coincide with it.

7.3. **The substitution lemma.**

Lemma 7.3 (Substitution/environment coherence)**.** *For every term t, substitution σ and environment ρ: if either of* $\mathsf{eval}(t\sigma, \rho)$, $\mathsf{eval}(t, \mathsf{eval}(\sigma, \rho))$ *is defined then both are, and*

$$\mathsf{eval}(t\sigma, \rho) \;\approx\; \mathsf{eval}(t, \mathsf{eval}(\sigma, \rho)).$$

Proof. Structural induction on t. All cases except transport are the standard NbE compositionality cases: value formers produce equal heads with $\approx$ arguments; the closure-forming cases are the closure clause of Definition 5.1 (instantiating either closure continues with the induction hypothesis); interval parts are literally equal by the depth-determinacy of fresh levels.

For $t = \mathsf{transp}^i A\, \varphi\, u$: the two family values

$$F_L := \mathsf{eval}(A\sigma, \rho)[\ell], \qquad F_R := \mathsf{eval}(A, \mathsf{eval}(\sigma, \rho))[\ell]$$

satisfy $F_L \approx F_R$ (induction hypothesis for A plus inst-compatibility), with the *same* fresh ℓ (depth determinacy). The check is the specification of Definition 6.1, which is $\approx$-invariant (Lemma 5.2); *hence the two sides take the same branch.* Positive branch: induction hypothesis for u. Negative branch: the same structural dispatch on the same head, closing by the induction hypotheses and the per-operation compatibility lemmas. □

Remark 7.4. The proof shows why the check *must* be specified representation-independently: an implementation-level check (a traversal of representations) is not a function of the $\approx$-class — the two sides of the lemma present the same family in different representations, and a representation-sensitive check could take different branches, breaking the lemma at exactly the transport case. Specification vs. implementation is not pedantry here; it is the load-bearing design decision. (For the artifact no gap remains: the implemented checker decides the specification exactly, Theorem 6.3 — and Remark 6.4 records how close it once came to having one.)

7.4. **The fundamental theorem, and termination.**

Definition 7.5 (Semantic typing)**.** $\Gamma \vDash t : A$ iff for all depths and all pairs of related Γ-environments $\rho \equiv^{\mathrm{val}} \rho'$: $\mathsf{eval}(t, \rho)$ is *defined* (evaluation terminates) and $\mathsf{eval}(t, \rho) \equiv^{\mathrm{val}}_{\mathsf{eval}(A,\rho)} \mathsf{eval}(t, \rho')$.

Theorem 7.6 (Fundamental theorem)**.** *If $\Gamma \vdash t : A$ in T_{alg} then $\Gamma \vDash t : A$.*

Proof. Induction on the typing derivation. All T_0 rules are the base component (B), in environment form (the enumeration, with the clause-by-clause correspondence to [4], is Appendix C). Two cases are specific to T_{alg}.

The conversion rule. From $\Gamma \vdash t : B$ and $\Gamma \vdash B \equiv_{\mathrm{alg}} A$: by IH, $\mathsf{eval}(t, \rho)$ is defined and related at $\mathsf{eval}(B, \rho)$; conversion correctness (Lemma 5.4(iii)) turns the accepted conversion

into $\mathsf{eval}(B, \rho) \equiv^{\mathrm{val}} \mathsf{eval}(A, \rho)$, and $\equiv^{\mathrm{val}}$ is invariant under related type values (Lemma L10), closing the case. This is where defining the system of record with the *algorithmic* equality is essential and not cosmetic: had the conversion rule used derivability in T_{eq}, this case would demand invariance of the predicates along (†)-type equations, which the algorithm refutes — the fundamental theorem would be false as stated.

The transport clause of the prioritized evaluator:

- *Positive branch.* By Theorem 6.2 the endpoint type values decode to the same PER — indeed they have equal read-back — so the induction hypothesis for u gives exactly the required relatedness at the result type. Note no closure property of the predicates under judgmental equality is invoked; the equality of PERs is by read-back identity. This is where the specification (Definition 6.1) pays a second time.
- *Negative branch.* The dispatch clauses are the base proof's structural transport cases. Their inner recursive transports (along domain lines, cod fibers, tube lines) are at *structurally smaller semantic type values*, so they are discharged by the inner induction on type values by which $\equiv^{\mathrm{val}}$ and the base transport case are defined — *not* by the outer induction on typing derivations. The proof is thus a double induction: outer on the derivation, inner on type values; the evaluator's own recursion is mirrored by the inner induction, which is well-founded by the stratified definition of $\equiv^{\mathrm{val}}$ (Definition 5.3). Termination of the dispatch is part of the conclusion, not an input.

□

Corollary 7.7 (Termination)**.** *Evaluation, the constancy check, conversion and read-back terminate on well-typed inputs.*

Proof. Termination of eval is part of Definition 7.5 and hence of Theorem 7.6. The check traverses a value that is semantically well-typed; its binder instantiations are instantiations of computable closures at generic arguments, which terminate by the same theorem; system closures are walked without instantiation. conv and quote recurse along the stratified type-value structure. No syntactic measure is claimed anywhere: this is the standard normalization-proof route [4, 5], and the only one available, since closure instantiation re-enters evaluation (Section 5.1). □

7.5. **Admissibility of the algorithmic equality.**

Theorem 7.8 (Admissible conversion relation)**.** *Relative to the base component (B), the typed algorithmic equality of Definition 7.1:*

(1) *is an equivalence relation on well-typed terms (reflexivity from Theorem 7.6 at the generic environment; symmetry and transitivity from the PER structure via Lemma 5.4(iii));*
(2) *is a congruence: for every term former, relatedness of the immediate subterms implies relatedness of the whole (each former evaluates to a value former whose $\equiv^{\mathrm{val}}$-clause is componentwise; the former-by-former list, including both transport branches, is Appendix C);*
(3) *is stable under weakening (Kripke monotonicity of $\equiv^{\mathrm{val}}$ in the depth);*
(4) *is stable under substitution: $\Gamma \vdash t \equiv_{\mathrm{alg}} u : A$ and $\Delta \vdash \sigma : \Gamma$ imply $\Delta \vdash t\sigma \equiv_{\mathrm{alg}} u\sigma : A\sigma$ (Lemma 7.3 composed with Lemma 5.4(i,iii) and Theorem 7.6 applied to σ);*
(5) *is stable under context conversion ($\Gamma \equiv_{\mathrm{alg}} \Gamma'$ pointwise implies the generic environments are $\equiv^{\mathrm{val}}$-related, and $\equiv^{\mathrm{val}}$ is invariant under related type values);*

(6) *preserves and respects typing in* T_{alg}*: if* $\Gamma \vdash t : A$ *and* $\Gamma \vdash A \equiv_{\mathrm{alg}} B$ *then* $\Gamma \vdash t : B$ *(this* is T_{alg}*'s conversion rule, and the fundamental theorem shows it harmless), and the bidirectional checker's conversion sites decide exactly this relation.*

Theorem 7.8 is the license to call the algorithmic equality a *judgmental* equality: it has every structural property the conversion rule of a dependent type theory requires.

8. Soundness, and what completeness would mean

Lemma 8.1 (Step soundness)**.** *Write* $\hat{\rho}$ *for the substitution quoting an environment pointwise, and* $t[\hat{\rho}]$ *for the result of applying it. If* $\mathsf{eval}(t, \rho)$ *is defined with value* v*, then* $t[\hat{\rho}] \equiv \mathsf{quote}(v)$ *is derivable in* $T_{\mathrm{eq}} = T_0 + (\mathrm{R})$*; and if* $\mathsf{conv}(v, v', T)$ *accepts then* $\mathsf{quote}(v) \equiv \mathsf{quote}(v')$ *is derivable in* T_{eq}*.*

Proof. Induction on the big-step derivation (Section 7.1), with the read-back statements of the sub-computations supplied by the same induction. Each evaluator rule is matched by a T_{eq} equation: β-steps and projections by the β rules; face collapses by the boundary rules; the structural transport dispatches by the corresponding structural rules of Section 2.1, instantiated at the quoted premises; hcomp steps by the composition rules; η-directed conversion clauses by the η rules, and interval comparisons by the free De Morgan axioms (sound for T_{eq}, which contains them).

The one step needing care is the *positive transport branch*: $\mathsf{eval}(\mathsf{transp}^i A\, \varphi\, u, \rho) = \mathsf{eval}(u, \rho)$ when $\mathsf{const?}_{\mathrm{spec}}(F, \ell)$ for F the family value at the fresh ℓ. We must exhibit a T_{eq} derivation of $(\mathsf{transp}^i A\, \varphi\, u)[\hat{\rho}] \equiv u[\hat{\rho}]$. Take $A_0 := \mathsf{quote}(F)$, a term not containing (the variable corresponding to) ℓ, by the definition of $\mathsf{const?}_{\mathrm{spec}}$. By the induction hypothesis applied to the family's evaluation and instantiation, $A[\hat{\rho}] \equiv A_0$ is derivable in T_{eq} (in the context extended by ℓ). So $A[\hat{\rho}]$ is *judgmentally* i*-constant* in the sense of Definition 3.1 — the premise of rule (R) — with witness A_0, and (R) yields $(\mathsf{transp}^i A\, \varphi\, u)[\hat{\rho}] \equiv u[\hat{\rho}]$. The induction hypothesis for u finishes the step. □

Theorem 8.2 (Soundness)**.** *Algorithmic equality is contained in derivability in* T_{eq}*: if* $\Gamma \vdash t \equiv_{\mathrm{alg}} u : A$ *then* $\Gamma \vdash t \equiv u : A$ *is derivable in* T_{eq}*.*

Proof. By Definition 7.1, conv accepts the values of t and u at the generic environment; Lemma 8.1 gives T_{eq}-derivations of $t \equiv \mathsf{quote}(\mathsf{eval}\, t)$, $u \equiv \mathsf{quote}(\mathsf{eval}\, u)$ (the generic environment quotes to the identity substitution) and $\mathsf{quote}(\mathsf{eval}\, t) \equiv \mathsf{quote}(\mathsf{eval}\, u)$; compose by transitivity. □

The asymmetry between Theorem 8.2 and Remark 8.3 below is the paper's central design fact: T_{alg}'s equality is *strictly weaker* than T_{eq}'s (the no-go equation separates them), yet every algorithmic step is T_{eq}-sound; the algorithm selects a consistent, substitution-stable sub-theory of an equational theory that is itself unusable.

Remark 8.3 (No completeness — necessarily)**.** By Theorem 4.6(1), $T_0 + (\mathrm{R})$ derives (†), which conv refutes; the checker is incomplete for $T_0 + (\mathrm{R})$, provably and by design. The system of record is the algorithmic equality itself — legitimately so, by Theorem 7.8 — and $T_0 + (\mathrm{R})$ is its sound equational over-approximation. We therefore do not use the phrase "decision procedure" for judgmental equality anywhere: the algorithmic equality is decidable by running its terminating checker (Corollary 7.7), and is soundly included in $T_0 + (\mathrm{R})$; nothing more is claimed.

9. Canonicity

Theorem 9.1 (Canonicity)**.** *Relative to the base component (B), in both the Cartesian and the De Morgan presentations — in the Cartesian fragment the judgmental-theory part of (B) follows from* [5]*, the evaluator-level bridge remaining open (Appendix G): every closed T_{alg}-term $t : \mathbb{N}$ evaluates to a numeral; the algorithmic equality is consistent ($0 \not\equiv_{\mathrm{alg}} 1$); and $\mathsf{J}\, d\, \mathsf{refl} \equiv_{\mathrm{alg}} d$.*

Proof. The closed instance of Theorem 7.6: the empty environment is trivially related to itself, so $\mathsf{eval}(t) \equiv^{\mathrm{val}}_{\mathbb{N}} \mathsf{eval}(t)$, and the $\equiv^{\mathrm{val}}_{\mathbb{N}}$ clause forces a numeral (there are no closed neutrals); numeral inversion is Appendix C. Consistency: 0 and 1 evaluate to distinct numerals, and conv separates them. For J: substituting $p := \mathsf{refl}$ makes the transport family's value read back without the transport dimension, so $\mathsf{const}?_{\mathrm{spec}}$ holds and the positive branch returns d's value; machine-checked end to end (`strictJReflD`). □

Corollary 9.2 (Unconditional minimal core)**.** *Over the minimal fragment $\Pi, \Sigma, \mathsf{Path}, \mathbb{N}$ with* transp *and* hcomp *(no* Glue*, no universes, no HITs), Theorems 7.6, 7.8 and 9.1 hold* with no base-component assumption*: Appendix F constructs the logical relation for the fragment by structural recursion on syntactic types and proves every (B)-marked lemma for it, former by former. In particular $\mathsf{J}\, d\, \mathsf{refl} \equiv_{\mathrm{alg}} d$ — J being defined from paths, connections and transport alone — is unconditional (Proposition F.29).*

Inheritance inventory. Reused from [4] (as component (B)): the predicate clauses for base formers, hcomp-computability (the composition clauses never inspect transport redexes), read-back correctness, numeral inversion. New in this paper: representation equivalence and its compatibility lemmas (Section 5.2); the check specification, its soundness (Theorem 6.2) and the checker-exactness theorem (Theorem 6.3); the substitution lemma through the transport case (Lemma 7.3); the positive transport case of the fundamental theorem; the explicit double-induction discharge of the negative case; and the admissibility theorem (Theorem 7.8).

10. The strictness catalogue

Machine-checked excerpt (positive rows: an inline refl witness is accepted; negative rows: the analogous witness is rejected; all rows are claims about the one algorithmic equality — implementation and specification coincide by Theorem 6.3):

law	holds?
connection reparametrizations (idempotence, involution, absorption, $i \wedge 0$)	yes
De Morgan duality square	yes
η for Π, Σ, Path	yes
$\mathsf{symm}\,\mathsf{refl} \equiv \mathsf{refl}$; $\mathsf{symm} \circ \mathsf{symm} \equiv \mathrm{id}$; cong laws	yes
transport along constant families (syntactic and value-level; at $\mathcal{U}$; at path families)	yes
switchover contracta at Π, Σ, dependent Σ, argument-wise HIT constructors (incl. merid)	yes
$\mathsf{J}\, d\, \mathsf{refl} \equiv d$	**yes**
$\mathsf{transport}\,(\mathsf{ua}\,\mathsf{idEquiv})\, x \equiv x$ (via $\mathsf{transpGlue}$)	yes
$\mathsf{trans}\,\mathsf{refl}\,\mathsf{refl} \equiv \mathsf{refl}$ (hcomp-regularity)	no
path switchover contractum (†)	no
$\mathsf{cong}/\mathsf{symm}$ distribution over trans	no
$\langle i \rangle\, p(i \wedge \neg i) \equiv \mathsf{refl}$ (Appendix B)	no

11. Implementation and artifact

The kernel is a self-contained CCHM-style implementation of about five thousand lines of Lean 4 (toolchain `4.31.0`), independent of Lean's own definitional equality: the NbE core with defunctionalized closures, bidirectional type checking, the De Morgan interval with the antichain-DNF decision procedure, Glue/univalence with transpGlue, and the higher inductive types of Section 4.2. The prioritized transport of Section 7 is the evaluator's transport clause; the occurs-check is implemented as described in Section 6.3. A ∼300-definition object-language library exercises the rule throughout ($\pi_1(S^1) \cong \mathbb{Z}$, untruncated Eckmann–Hilton and Mac Lane coherence, a groupoid classifying-space HIT with its classification theorem), validated by a differential harness fingerprinting the normal forms of every library definition.

Artifact and its scope. The repository — kernel, catalogue (`LibStrictness.lean`), switchover experiments (`LibSwitchover.lean`), library, harness, and a row-by-row paper-to-probe correspondence (`ARTIFACT.md`) — is public at

`https://github.com/r-shibusawa/cubical-strict-j`

release `v1.0.0`, commit `6c5904b` (archived snapshot: DOI to be inserted); pinned toolchain Lean `4.31.0`; build with `lake build`: every catalogue and switchover claim is an elaboration-time guard, so a successful build is the verification run (continuous integration replays it on every commit). We distinguish explicitly: the artifact provides *machine-checked object-language witnesses* (all algorithmic-equality claims) and *executable regression tests* (the differential harness); it does *not* provide machine-checked metatheory — the theorems of Sections 4–9 are pen-and-paper results about the specified system, which the kernel implements exactly (Theorem 6.3); a kernel passing its test suite remains evidence, not proof, of its own soundness.

12. Related work

Constancy formulas. The φ-discipline is from ABCFHL [2]; Cubical Agda [3] fixes φ at elaboration time, whence the propositional transportRefl. Rule (R) is the conversion-closure of ($\mathrm{S}_{\mathsf{i}1}$); Remark 3.2 locates exactly when this is a proper extension.

Regularity. CCHM [1] initially claimed regular composition; the universe error was found by Sattler [7], and Swan [6] separated path and identity types in presheaf models. Those results concern hcomp/composition; Theorem 4.6 connects them to transport-level constancy inference: prioritization is exactly what prevents the equational collapse onto hcomp-regularity. Recent work towards computational UIP in Cubical Agda [9] documents the practical cost of missing regularity.

Systems with regular coercion. Arend [8] has a regular coe over a non-cubical interval; the folklore obstruction to porting this to full cubical theories — which Theorem 4.6 makes precise — was the interaction with structural transport and Glue. XTT and observational theories [12] obtain a strict J by strengthening equality globally, at the cost of the univalent universe; the present system keeps the ABCFHL setting unchanged.

Normalization and NbE metatheory. The logical relation of Section 5 is the standard Kripke PER structure of NbE correctness proofs, transposed to the cubical domain; the environment-based fundamental theorem follows Huber [4], and the Cartesian discharge of the base component is Sterling–Angiuli [5] (normal-form specification in [10]). What is new is confined to the transport clause: the representation-independent check specification, its soundness theorem, and the resulting substitution and admissibility results.

13. Conclusion and open problems

Evaluation-time constancy inference is an old idea. The analysis here shows why its naive equational formulations could not work — they derive constant-system hcomp-regularity, at path types already (Theorem 4.6) — and that the operational formulation, done with a representation-independent check specification, supports the full structural metatheory expected of a judgmental equality (Theorem 7.8) together with termination and canonicity (Theorem 9.1), and computes J on refl — *unconditionally on the minimal core* $\Pi, \Sigma, \mathsf{Path}, \mathbb{N}$ (Theorem 9.2, Proposition F.29), and *relative to the delimited base component* for the full implemented univalent calculus. We do not claim an unconditional strict J for the full univalent system; the three-layer division of Section 1 is the paper's claim, exactly.

Open problems, in decreasing order of importance. (1) *The base component beyond the core*: for the core fragment of Appendix F the base component is discharged in this paper and the positive results are unconditional; beyond it, the judgmental-theory part of the Cartesian base component follows from [5], while the concrete evaluator-level bridge (Appendix G), universes, Glue and the wider HIT roster remain to be discharged — on the De Morgan side, in full. (2) *A semantic countermodel for* (†): upgrading Theorem 4.6(2) from "closely related to excluded principles" to a refutation in a concrete cubical-sets model. (3) *Model-theoretic status of the positive system*: whether a cubical-sets-style model validates the specified algorithmic equality. (4) *Characterization*: is there any rule-generated presentation of the operational system? We conjecture not. (5) *Boundary-corrected HIT signatures*: extending the switchover classification of Section 4.2 beyond the argument-wise class.

Acknowledgements. Three rounds of critical review of earlier drafts materially reshaped this paper: the observation that non-constancy is not substitution-stable prompted the switchover experiments that became Theorem 4.6, and the demand for the structural metatheory of the algorithmic equality prompted Sections 5–9 in their present form.

Appendix A. The base calculus in full

This appendix fixes T_0 completely: grammar, judgments, typing, and the equational rules, matching the artifact's kernel former by former. ($\Gamma \vdash \varphi\ \mathsf{cof}$ premises and routine congruence rules are elided from displays.)

A.1. **Grammar and judgments.**

$$\begin{aligned} r, s &::= i \mid 0 \mid 1 \mid r \wedge s \mid r \vee s \mid \neg r \qquad\qquad \varphi, \psi ::= (r{=}0) \mid (r{=}1) \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \mathsf{i0} \mid \mathsf{i1} \\ A, B, t, u &::= x \mid \mathcal{U}_k \mid \Pi x{:}A.B \mid \lambda x.t \mid t\,u \mid \Sigma x{:}A.B \mid (t,u) \mid \mathsf{fst}t \mid \mathsf{snd}t \\ &\quad \mid \mathsf{PathP}\,(\langle i\rangle A)\,t\,u \mid \langle i\rangle t \mid t\,r \mid \mathbb{N} \mid \mathsf{ze} \mid \mathsf{su}\,t \mid \mathsf{natrec} \\ &\quad \mid \mathsf{transp}^i A\,\varphi\,u \mid \mathsf{hcomp}^i A\,[\overrightarrow{\varphi \mapsto t}]\,u \mid \mathsf{Glue}\,[\overrightarrow{\varphi \mapsto (T,w)}]\,B \\ &\quad \mid \mathsf{glue} \mid \mathsf{unglue} \mid \text{HIT formers (Section 4.2)} \end{aligned}$$

Judgments: $\Gamma\ \mathsf{ctx}$; $\Gamma \vdash A\ \mathsf{type}_k$ (equivalently $\Gamma \vdash A : \mathcal{U}_k$, universes à la Russell, cumulative); $\Gamma \vdash t : A$; $\Gamma \vdash \varphi\ \mathsf{cof}$; and the *algorithmic* equality $\Gamma \vdash t \equiv_{\mathrm{alg}} u : A$ of Definition 7.1, used in the conversion rule. Contexts extend by term variables, interval variables and cofibration restrictions. Interval expressions are identified up to free De Morgan equality (decided by antichain-DNF); the implementation additionally provides $\mathsf{Int}, \mathsf{Unit}, \mathsf{Empty}$, binary sums and lists, whose rules are routine copies of the $\mathbb{N}$ pattern.

A.2. **Structural core.**

$$\frac{\Gamma, x{:}A \vdash t : B}{\Gamma \vdash \lambda x.t : \Pi x{:}A.B} \qquad \frac{\Gamma \vdash t : \Pi x{:}A.B \quad \Gamma \vdash u : A}{\Gamma \vdash t\,u : B[u/x]} \qquad (\beta,\ \eta_\Pi :\ \lambda x.\,t\,x \equiv t)$$

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B[t/x]}{\Gamma \vdash (t,u) : \Sigma x{:}A.B} \qquad (\mathsf{fst}/\mathsf{snd}\text{ projections};\ \beta;\ \eta_\Sigma :\ (\mathsf{fst}t, \mathsf{snd}t) \equiv t)$$

$$\frac{\Gamma, i{:}\mathbb{I} \vdash t : A}{\Gamma \vdash \langle i\rangle t : \mathsf{PathP}\,(\langle i\rangle A)\,t(\mathsf{i0})\,t(\mathsf{i1})} \qquad \frac{\Gamma \vdash p : \mathsf{PathP}\,(\langle i\rangle A)\,l\,r \quad \Gamma \vdash s : \mathbb{I}}{\Gamma \vdash p\,s : A(s/i)}$$

$$(\beta;\ p\,\mathsf{i0} \equiv l,\ p\,\mathsf{i1} \equiv r;\ \eta_{\mathsf{Path}} :\ \langle i\rangle\,p\,i \equiv p)$$

$$\frac{}{\Gamma \vdash \mathbb{N} : \mathcal{U}_0} \qquad \mathsf{natrec}\text{ with its two computation rules} \qquad \frac{\Gamma \vdash A : \mathcal{U}_k}{\Gamma \vdash A : \mathcal{U}_{k+1}}$$

A.3. **Kan operations.** Transport was displayed in Section 2.1: the typing rule with explicit φ, the collapse rule (S_{i1}), and the structural rules for Π, Σ, Path. The remaining structural transport rules: at $\mathcal{U}_k$, $\mathbb{N}$ and base types, $\mathsf{transp}^i A\,\varphi\,u \equiv u$ (the line is constant by typing); at HIT formers, argument-wise per Section 4.2; at Glue, the $\mathsf{transpGlue}$ unfolding of [1, §6.2] (its constant-family degeneration is computed in Section 4.4); on neutral lines, transport is neutral.

Homogeneous composition:

$$\frac{\Gamma \vdash A\ \mathsf{type} \qquad \Gamma \vdash u : A \qquad \Gamma, \varphi_k, i{:}\mathbb{I} \vdash t_k : A\ \ (\forall k)}{\Gamma \vdash \mathsf{hcomp}^i A\,[\overrightarrow{\varphi_k \mapsto t_k}]\,u : A}$$

with side conditions $\Gamma, \varphi_k \vdash t_k(\mathsf{i0}) \equiv u$ and $\Gamma, \varphi_k \wedge \varphi_l, i \vdash t_k \equiv t_l$, with $\Gamma, \varphi_k \vdash \mathsf{hcomp}\,[\cdots]\,u \equiv t_k(\mathsf{i1})$, and $\mathsf{hcomp}\,[\mathsf{i1} \mapsto t]\,u \equiv t(\mathsf{i1})$; structural rules push hcomp into $\Pi/\Sigma/\mathsf{Path}$ components, compute at $\mathbb{N}$ and constructors, and are constructors at HITs (hcomp-cells). *No* constancy rule for hcomp exists (Theorem 4.6 is why none may be added). $\mathsf{comp}^i A\,[\vec{\varphi} \mapsto \vec{t}]\,u := \mathsf{hcomp}^i A(\mathsf{i1})\,[\vec{\varphi} \mapsto \mathsf{transp}\text{-filled }\vec{t}]\,(\mathsf{transp}^i A\,\mathsf{i0}\,u)$.

A.4. **Glue.**

$$\frac{\Gamma \vdash B : \mathcal{U}_k \quad \Gamma, \varphi \vdash T : \mathcal{U}_k \quad \Gamma, \varphi \vdash w : \mathsf{Equiv}\, T\, B}{\Gamma \vdash \mathsf{Glue}\,[\varphi \mapsto (T, w)]\, B : \mathcal{U}_k} \qquad \Gamma, \varphi \vdash \mathsf{Glue}\,[\varphi \mapsto (T, w)]\, B \equiv T$$

with glue/unglue introduction and elimination, their β rule, the face collapses, and univalence realized as ua by gluing along an equivalence; Equiv is the standard contractible-fibers definition (no universe needed, as in [5]).

A.5. **Conversion rule.**

$$\frac{\Gamma \vdash t : A \qquad \Gamma \vdash A \equiv_{\mathrm{alg}} B}{\Gamma \vdash t : B}$$

The equality in the conversion rule is the algorithmic equality — legitimately, by Theorem 7.8; the bidirectional checker of the artifact implements exactly this rule at its switching points.

Appendix B. Reparametrization coherence

A self-contained result delimiting the strict/weak boundary from below: reparametrizations are strict, and whatever generates an hcomp is not — as Theorem 4.6 explains, this is no accident. Fix the generic context

$$\Gamma = (A : \mathcal{U},\ a\, b : A,\ p : \mathsf{Path}\, A\, a\, b).$$

Theorem B.1 (Reparametrization coherence)**.** *Let $\varphi, \psi \in \mathrm{DM}(i_1, \ldots, i_n)$ agree at every corner $\varepsilon \in \{0,1\}^n$. Then*

$$\langle i_1 \cdots i_n \rangle\, p(\varphi) \ \equiv_{\mathrm{alg}}\ \langle i_1 \cdots i_n \rangle\, p(\psi) \qquad \Longleftrightarrow \qquad \varphi = \psi \ \textit{in}\ \mathrm{DM}(i_1, \ldots, i_n).$$

Proof. ($\Leftarrow$) Conversion instantiates binders at generic dimensions $\vec{\ell}$; $i_k \mapsto \ell_k$ is a De Morgan homomorphism, and neutral path applications compare interval arguments by canonical antichain-DNF, sound and complete for free De Morgan equality; if the common value is $0/1$, both sides collapse to the same endpoint. ($\Rightarrow$) In the generic context p, a, b are distinct neutrals. If neither side collapses, DNF equality of $\varphi(\vec{\ell})$ and $\psi(\vec{\ell})$ is demanded, and generators map to generators, so $\varphi = \psi$. If exactly one side collapses, a variable is compared against a neutral application and conversion fails. If both collapse, the corner hypothesis forces the same endpoint. □

In particular $\langle i \rangle\, p(i \wedge \neg i) \not\equiv_{\mathrm{alg}} \mathsf{refl}_a$: the absence of excluded middle in the free De Morgan algebra is reflected in the algorithmic equality (machine-checked, both directions).

Appendix C. Lemmas: statements and proofs

This appendix carries out the routine inductions behind Sections 5–9. A structural remark organizes everything:

Remark C.1 (The domain is unchanged)**.** Rule (R) alters the *evaluator*, not the *domain*: it prunes transport redexes and introduces no new value forms. The values, neutrals, closures, read-back and conversion of the extended system are literally those of the base system. Consequently every base lemma *about values* — the PER laws, Kripke monotonicity, conversion correctness, read-back correctness, numeral inversion — transfers verbatim; the extension is audited entirely in (i) the evaluator clauses (the fundamental theorem, Appendix D) and (ii) the check (Section 6). Lemmas below marked (B) are of the first kind

and constitute the base component of Section 2.2; lemmas marked (new) are of the second kind and are proved here in full.

C.1. **Depth and freshness.**

Lemma C.2 (L1: depth invariant (new))**.** *Every free level of a value produced by evaluation at depth d is $< d$; the level bound* $\mathrm{lb}(v)$ *maintained by the smart constructors satisfies* $\mathsf{FV}(v) \subseteq [0, \mathrm{lb}(v))$ *and* $\mathrm{lb}(v) \leq d$.

Proof. Induction on the evaluation step producing v. Environment entries were produced at outer depths, so the invariant holds for them by hypothesis; a fresh level is allocated as the current depth d and only inside a binder, i.e. at inner depth $d{+}1$, where $d < d{+}1$; every smart constructor sets lb := max of its parts' bounds, and no operation manufactures a level except fresh allocation. The level-bound cut of Section 6.3 ($\ell \geq \mathrm{lb}(v) \implies \ell \notin \mathsf{FV}(v)$) is immediate. □

Lemma C.3 (L2: depth determinacy (new))**.** *Let two evaluation runs traverse the same term spine at the same depth (in possibly different environments). Then they allocate the same fresh levels at the same positions.*

Proof. Fresh levels are, by construction, *equal to the depth at allocation*; depth evolves only by crossing a binder of the term spine being traversed, which is the same on both runs. (This is the reason Definition 5.1 may demand *literal* equality of interval parts: the interval values produced by two $\approx$-related computations are built from the same generic levels by the same De Morgan operations.) □

C.2. **Compatibility of representation equivalence.** All compatibility lemmas are proved by one simultaneous coinduction: the relation

$$\mathcal{R} \;:=\; \{\, (\mathsf{op}(\vec{v}),\ \mathsf{op}(\vec{v}')) \mid \mathsf{op} \text{ a semantic operation},\ \vec{v} \approx \vec{v}' \,\}$$

is shown to be a $\approx$-bisimulation, by case analysis on op and on the (common) head of the principal arguments. We record the cases separately for reference.

Lemma C.4 (L3: environments and instantiation (new))**.** *If $\rho \approx \rho'$ pointwise then* $\mathsf{eval}(t, \rho) \approx \mathsf{eval}(t, \rho')$*; consequently if moreover $c \approx c'$ and $w \approx w'$ then* $\mathsf{inst}_d(c, w) \approx \mathsf{inst}_d(c', w')$.

Proof. For mk-closures, inst is an eval call on the common body in $\approx$-related environments; for each closure combinator, inst is a fixed composite of semantic operations applied to the stored fields, and the fields are $\approx$ by assumption. The eval statement is by induction on t: value formers yield equal heads with $\approx$ arguments (interval parts literal by Lemma C.3); closure formers store $\approx$-related environments over the same body, which is exactly the closure clause of Definition 5.1; eliminators reduce to L4–L8. □

Lemma C.5 (L4: application, projections, path application (new))**.** $\approx$ *is preserved by* app*,* fst*,* snd*,* papp.

Proof. Case on the common head. β-heads (lam, pair, plam): the operation is an instantiation or a field projection — L3, or immediate. Neutral heads: the spine grows by $\approx$-related arguments, which is the neutral clause. Path application at an endpoint returns a stored endpoint field — immediate. □

Lemma C.6 (L5: face forcing (new))**.** $\approx$ *is preserved by* force *(deciding faces of* hcomp*/*glue *values under a cofibration valuation).*

Proof. The decision inspects cofibration arguments, which are literally equal; both sides therefore decide identically, and the surviving branches are $\approx$ by assumption. □

Lemma C.7 (L6: transport (new))**.** *If $c \approx c'$ (family closures) and $u \approx u'$ then* transp*-evaluation on (c, u) and (c', u') yields $\approx$-related results.*

Proof. The instantiated family values $F := \mathsf{inst}_{d+1}(c, \ell)$ and $F' := \mathsf{inst}_{d+1}(c', \ell)$ are $\approx$ (L3; same ℓ by L2). The check is $\mathsf{const?}_{\mathrm{spec}}$, invariant under $\approx$ by Lemma 5.2: *both sides take the same branch.* Positive branch: $u \approx u'$. Negative branch: same head of F, F'; each structural dispatch is a fixed composite of L3–L5, L7, L8 and recursive L6-instances on stored fields, closing the bisimulation. □

Lemma C.8 (L7: homogeneous composition (new))**.** $\approx$ *is preserved by* hcomp *and by system formation.*

Proof. System entries are pairs of literally-equal cofibrations and $\approx$-related closures (L3); if a face is decided, both sides decide it identically (L5) and continue in the tube; otherwise both form an hcomp value with componentwise-$\approx$ arguments. □

Lemma C.9 (L8: Glue operations (new))**.** $\approx$ *is preserved by* glue*,* unglue *and* transpGlue*.*

Proof. glue/unglue: head formation and field extraction over componentwise-$\approx$ data, with face decisions by L5. transpGlue is a fixed composite of instantiations, transports, compositions, projections and (un)gluings ([1, §6.2]); each step is L3–L7 or a recursive L6-instance. □

(L9 is Lemma 5.2 in the main text.)

C.3. **Value-level lemmas of the base component.** By Remark C.1 the following are statements about the shared domain and transfer verbatim from the base theory; we record the statements and the proof shapes for self-containment.

Lemma C.10 (L10: PER laws and type invariance (B))**.** *Each $\equiv^{\mathrm{val}}_T$ is symmetric and transitive, and if $T \equiv^{\mathrm{val}}_{\mathcal{U}} T'$ then $\equiv^{\mathrm{val}}_T = \equiv^{\mathrm{val}}_{T'}$.*

Proof shape. Induction on the stratified type value. Symmetry and transitivity are inherited clause-wise (e.g. at Π from the codomain PERs at related arguments, using symmetry to swap and transitivity to compose through a common argument). Type invariance: related type values have, clause by clause, cofinally the same decompositions (related domains and codomain families), so the defined PERs coincide; at $\mathcal{U}$ this is the universe induction. □

Lemma C.11 (L11: Kripke monotonicity (B))**.** *$v \equiv^{\mathrm{val}}_T v'$ at depth d and $d' \geq d$ imply $v \equiv^{\mathrm{val}}_T v'$ at depth d'.*

Proof shape. The quantifiers in Definition 5.3 range over all future depths; restriction is monotone. □

Lemma C.12 (L12: conversion correctness (B))**.** $\mathsf{conv}_d(v, v', T)$ *accepts iff $v \equiv^{\mathrm{val}}_T v'$. The extension adds* no *clause to* conv*: conversion consumes values, and rule (R) only changes which values the evaluator produces.*

Proof shape. Soundness: induction on the accepting run; each clause of conv (η-directed at $\Pi/\Sigma/\mathsf{Path}$, DNF equality for interval arguments of neutral path applications, spine comparison for neutrals, componentwise elsewhere) verifies a defining clause of $\equiv^{\mathrm{val}}$. Completeness: by L13, related values have equal read-back, and conv accepts values with equal read-back (induction on the read-back). □

Lemma C.13 (L13: read-back correctness (B))**.** *$v \equiv^{\mathrm{val}}_T \mathsf{eval}(\mathsf{quote}_d\, v)$; distinct normal forms of the same type are inconvertible; read-back is injective on $\equiv^{\mathrm{val}}$-classes.*

Lemma C.14 (L14: read-back/action commutation (new))**.** *$\mathsf{quote}_d(v\langle\xi\rangle) = \mathsf{quote}_d(v)\langle\xi\rangle$ for any dimension substitution ξ whose domain is disjoint from the generic levels used by the read-back.*

Proof. Induction on the read-back. Head cases: the action preserves the head and acts on interval parts; read-back copies interval parts in DNF, and the action followed by DNF-renormalization equals DNF-renormalization followed by the syntactic substitution, because substitution into a De Morgan expression is a homomorphism and the antichain DNF is canonical. Binder cases: read-back opens the closure at a generic level $\notin \operatorname{dom}\xi$; the action passes into the closure environment (its definition) and the induction hypothesis applies to the opened body. Neutral cases: spine-wise. □

Lemma C.15 (L15: numeral inversion (B))**.** *A closed value v with $v \equiv^{\mathrm{val}}_{\mathbb{N}} v$ is a numeral.*

Proof shape. The $\equiv^{\mathrm{val}}_{\mathbb{N}}$ clause allows numerals or neutrals with equal read-back; closed neutrals do not exist (a neutral's head is a variable level, and depth 0 admits none). □

Appendix D. The fundamental theorem, case by case

Throughout, $\rho \equiv^{\mathrm{val}}_{\Gamma} \rho'$ abbreviates "related Γ-environments", and *IH* the induction hypothesis of the outer induction (on the typing derivation). "Defined" includes termination (Definition 7.5). Cases marked (B) are the base component in environment form; we give their proofs in the uniform one-paragraph style to make the claimed transposition inspectable, with the understanding that they follow [4] (De Morgan) — the correspondence for the Cartesian layer is Appendix G.

Variables (B).. $\mathsf{eval}(x, \rho) = \rho(x)$, defined and related by assumption on $\rho \equiv^{\mathrm{val}} \rho'$.

Π-introduction (B).. $\mathsf{eval}(\lambda x.t, \rho)$ is a closure value; for the Π-clause of $\equiv^{\mathrm{val}}$, take $d' \geq d$ and $w \equiv^{\mathrm{val}}_D w'$; instantiation continues as $\mathsf{eval}(t, \rho{+}[w])$, and $\rho{+}[w] \equiv^{\mathrm{val}} \rho'{+}[w']$, so IH for t applies; definedness of the body's evaluation under every related extension is exactly IH's definedness clause.

Π-elimination (B).. $\mathsf{eval}(t\,u, \rho) = \mathsf{app}(\mathsf{eval}(t, \rho), \mathsf{eval}(u, \rho))$; by IH both parts are defined and related; the Π-clause of $\equiv^{\mathrm{val}}$ applied at $w := \mathsf{eval}(u, \rho) \equiv^{\mathrm{val}} w' := \mathsf{eval}(u, \rho')$ gives definedness and relatedness of the application; the type is the instantiated codomain, matching the typing rule by L3.

Σ (B).. Introduction: componentwise by IH. Eliminations $\mathsf{fst}/\mathsf{snd}$: the Σ-clause is componentwise; projections of related values are related at domain and instantiated codomain respectively.

Path-introduction (B).. $\mathsf{eval}(\langle i\rangle t, \rho)$ is a path closure; the PathP-clause quantifies over interval values ι; instantiation continues as $\mathsf{eval}(t, \rho{+}[\iota])$ with IH; the endpoint conditions are the typing rule's boundary premises evaluated by IH, using L12 to convert the boundary equalities.

Path-elimination (B).. Application of related path values to a common interval value, by the PathP-clause; at endpoints the stored endpoint values are returned, related by the formation premises.

Universe rules (B).. Codes evaluate to type values; the $\mathcal{U}$-clause requires the decoded PERs to be related, which is IH at the smaller universe level (the stratification of Definition 5.3); cumulativity is monotone inclusion of the stratification.

$\mathbb{N}$ (B).. zero, suc: numeral clauses. Recursor: by the $\equiv^{\mathrm{val}}_{\mathbb{N}}$ clause the scrutinee is a numeral or neutral with equal read-back; in the numeral case compute by IH on the branch, by induction on the numeral; in the neutral case both sides form neutral recursor spines with related components, related by the neutral clause.

hcomp, all formers (B).. If a face of the system is decided, both evaluations continue in that tube at the top of the fill dimension (L5), related by IH on the system entry. Otherwise: at $\Pi/\Sigma/$Path the composition commutes into the components (the base structural hcomp rules), reducing to IH and the componentwise clauses; at base types and HITs the value is an hcomp-constructor covered by the hcomp-closure clause of $\equiv^{\mathrm{val}}$; at Glue the composite operation is a fixed composite of covered operations (L8). *No clause of this case mentions transport redexes* — the observation that lets the extension leave it untouched.

Glue (B).. Formation: componentwise (type values with related base and branch data on faces). Introduction (glue): the Glue-clause asks relatedness of the unglued part and of the branch parts on their faces — both by IH. Elimination (unglue): extraction, plus the decided-face case by L5.

HIT constructors and eliminators (B).. For the signatures of Section 4.2: constructors are covered by the constructor clause of $\equiv^{\mathrm{val}}$ (value arguments by IH, interval arguments literal); eliminators follow the recursor pattern: on constructor values compute by IH and the signature's computation rule; on hcomp values use the hcomp-closure clause and commute; on neutrals form related neutral spines.

Structural transport dispatches (B), and where the inner induction runs. These are the negative-branch continuations of the transport clause; we spell out the two used in Section 4 and the pattern for the rest.

- Π-*line.* The result is the closure value $\lambda x_1.\mathsf{transp}(\mathrm{cod}\ \mathrm{line})\,(f\,(\mathsf{transp}(\mathrm{dom}\ \mathrm{line})^{-}\,x_1))$. To place it in the Π-clause at the endpoint type, take related arguments $w \equiv^{\mathrm{val}} w'$ at the endpoint domain; the reversed domain transport is a transport along a line whose fibers are *structurally smaller type values* than the Π-line, so the *inner* induction (on type values, by which $\equiv^{\mathrm{val}}$ is defined) provides its definedness and relatedness; likewise the codomain transport. The outer IH is used only for the premises (f, the line). This is the division of labor claimed in Theorem 7.6: the evaluator's recursion along the line's structure is mirrored by the inner induction, never by the outer one.

- Σ-*line,* Path-*line, HIT lines,* Glue *line.* Same pattern: each inner transport or composition is at a structurally smaller type value (domain fiber, codomain fiber over a transported point, tube line, branch line), discharged by the inner induction; hcomp components by the hcomp case; transpGlue's steps by L8 and the inner induction.
- *Neutral line.* Both sides form related neutral transport spines (neutral clause).

Transport, positive branch (new). Suppose $\mathsf{const?}_{\mathrm{spec}}(F, \ell)$. By Theorem 6.2 the endpoint type values have equal read-back and decode to the same PER; by IH $\mathsf{eval}(u, \rho) \equiv^{\mathrm{val}} \mathsf{eval}(u, \rho')$ at the source endpoint, hence at the target endpoint *by identity of the PERs* — no closure property under judgmental equality is invoked. Definedness is IH's. By Lemma 7.3 and Lemma 5.2 the branch taken is a function of the $\approx$-class, so the case is stable under the environment/substitution manipulations used elsewhere in the proof.

Congruence lemmas (Theorem 7.8(2)). For each former, the lemma "subterms related $\Longrightarrow$ whole related" is the corresponding case above read with the IH replaced by the assumption; we list the non-obvious instances: transp-congruence needs the branch-agreement argument (the two families are $\equiv^{\mathrm{val}}$, hence — being values of well-typed families — of equal read-back by L13, so $\mathsf{const?}_{\mathrm{spec}}$ agrees); hcomp-congruence needs L5 to align face decisions; HIT-eliminator congruence needs the neutral clause when the scrutinee is neutral. All others are componentwise.

Appendix E. Checker exactness: the full clause table

Theorem 6.3 rests on the claim that `usesLvl` and quote have mirrored recursion structures. This appendix makes the claim inspectable. Throughout, "$\ell \in \mathsf{iv}(r)$" means the level ℓ occurs in the interval value r, and we use that interval quotation is *structural* (it maps the De Morgan operators of the value to the corresponding term formers), so $\ell \in \mathsf{iv}(r) \iff \ell \in \mathsf{FV}(\mathsf{quote}(r))$.

Definedness transfer. Both functions begin by forcing the value's head under the current cofibration valuation — the same force — and recurse along the same sub-computations: on value arguments structurally, and on closures via the *same* instantiation inst at the *same* fresh level (equal to the current depth, Lemma L2). Hence the big-step derivation trees of $\mathtt{usesLvl}(\ell, F)$ and $\mathsf{quote}(F)$ are isomorphic — one is defined iff the other is — with two exceptions in `usesLvl`'s favour, both of which only prune sub-derivations: the level-bound cut and the early exit of the disjunctive traversal (`usesLvl` may stop at the first occurrence). Pruning cannot destroy definedness.

Clause table. Value cases (heads after force):

head	usesLvl	quote
constants ($\mathcal{U}_k$, $\mathbb{N}$, 0, ...)	false	closed term
suc n, injections, tin, qin, ...	recurse on arguments	recurse on arguments
HIT cells (e.g. merid $a\,r$, emloop $g\,r$, squash cells)	args $\vee\ \ell \in \mathsf{iv}(r_i)$	args, quote(r_i)
Π/Σ (domain, codomain closure)	dom $\vee$ binder-inst.	dom, binder-inst.
λ-value (closure)	binder-inst.	binder-inst.
PathP (line closure, endpoints)	i-binder-inst. $\vee$ endpoints	i-binder-inst., endpoints
path abstraction (closure)	i-binder-inst.	i-binder-inst.
interval value r	$\ell \in \mathsf{iv}(r)$	quote(r)
Glue type / glue value	components $\vee$ cof mentions	components, cof terms
neutral	spine (below)	spine (below)

Neutral cases: variable (ℓ-equality of the head level $\leftrightarrow$ occurrence of the corresponding de Bruijn index in the quoted variable); application, projections, path application, recursors (spine pointwise, motive closures by binder instantiation); *neutral transport* (family by interval-binder instantiation, argument structurally); *neutral* hcomp (type and base structurally, each tube by cofibration mentions plus interval-binder instantiation of the tube closure — the case that the legacy dead code of Remark 6.4 would have treated structurally); unglue and the HIT eliminators (spine pointwise). In every row the right column's free-variable content is computed by the same recursion as the left column's disjunction, so the biconditional propagates by induction on the (isomorphic) derivations.

Completeness of the level-bound cut. The invariant of Lemma L1 is maintained by *every* smart constructor, including the closure formers: the bound of a closure is the maximum of the bounds of its stored environment entries and combinator fields. If $\ell \geq \mathrm{lb}(v)$, then ℓ is not among v's stored levels; levels introduced by instantiations during read-back are fresh, i.e. $\geq$ the current depth $> \ell$'s binding depth, and are bound by the binders the read-back produces. So $\ell \notin \mathsf{FV}(\mathsf{quote}(v))$: the cut answers exactly what the specification would.

Transparency of memoization. The memo hooks cache the result of $\mathtt{usesLvl}(\ell, \cdot)$ keyed by the level and the *physical identity* of the value (pointer equality). Physical identity implies structural identity, and usesLvl is a pure function of its arguments (it neither allocates observable state nor depends on any); caching a pure function at a sound key is observationally transparent. The generation-stamp discipline ensures a value reachable from two environments is the *same* value, so pointer keys do not conflate distinct values. Likewise the depth argument is determined by the value's stamp, so omitting it from the key loses nothing.

Appendix F. Discharging the base component for the minimal core

This appendix proves component (B) — and hence Theorems 7.6, 7.8 and 9.1 unconditionally (Corollary 9.2) — for the *minimal core*

$$T^{\mathrm{core}} \;:=\; \Pi,\ \Sigma,\ \mathsf{PathP},\ \mathbb{N} \text{ with } \mathsf{transp} \text{ and } \mathsf{hcomp} \quad (\text{no } \mathsf{Glue},\ \text{no universes},\ \text{no HITs}),$$

over the De Morgan interval; the Cartesian variant is a simplification of the same proof (drop the connection cases of the interval algebra). The fragment is chosen as the *smallest* one in which J is definable (Proposition F.29); the extension by S^1 is treated separately, as a proposition with a proof sketch and artifact evidence (Section F.11), precisely so that the headline theorem is not entangled with higher-inductive proof obligations.

F.1. **Types, values, normal forms, and the induction order.** Since there is no universe, types are a separate syntactic category:

$$T, S ::= \mathbb{N} \mid \Pi x{:}T.S \mid \Sigma x{:}T.S \mid \mathsf{PathP}\,(\langle i\rangle T)\,t\,u.$$

Lemma F.1 (Type-value inversion)**.** *If* $\mathsf{eval}(T,\rho)$ *is defined, its head is the value former matching* T*'s head former, and its closure fields store the syntactic subexpressions of* T *with (extensions of)* ρ*:* $\mathsf{eval}(\Pi x{:}T.S,\rho) = \mathsf{v}\Pi(\mathsf{eval}(T,\rho), \langle \rho; S\rangle)$*, and similarly for* Σ *and* PathP*.*

Proof. Immediate from the evaluator's type clauses: each is a single head-formation step storing a mk-closure. □

Remark F.2 (The induction order, precisely)**.** Lemma F.1 is what makes the fragment's metatheory elementary: every instantiation of a stored closure of a type value is an evaluation of a *proper syntactic subexpression* in an extended environment. Accordingly, the logical relation below is defined by *structural recursion on the syntactic type* T, carrying the environment as a parameter — when the Π-clause speaks of $\mathsf{inst}(C,w)$, it is, by Lemma F.1, speaking of $\mathsf{eval}(S, \rho{+}[w])$ with S a proper subterm of $\Pi x{:}T.S$. This is the exact well-founded order behind every "structurally smaller type value" step in this appendix; no ordinal, size measure on values, or universe stratification appears anywhere.

Normal and neutral forms of the core (the target of read-back):

$$\mathsf{nf} ::= \mathsf{ze} \mid \mathsf{su}\,\mathsf{nf} \mid \lambda x.\mathsf{nf} \mid (\mathsf{nf},\mathsf{nf}) \mid \langle i\rangle \mathsf{nf} \mid \mathsf{ne}$$

$$\mathsf{ne} ::= x \mid \mathsf{ne}\,\mathsf{nf} \mid \mathsf{fstne} \mid \mathsf{sndne} \mid \mathsf{ne}\,r \mid \mathsf{natrec}(\mathsf{nf},\mathsf{nf},\mathsf{nf},\mathsf{ne}) \mid \mathsf{transp}^{i}\mathsf{T}_{\mathsf{ne}}\,\varphi\,\mathsf{nf} \mid \mathsf{hcomp}\,[\vec{\varphi} \mapsto \vec{\mathsf{nf}}]\,\mathsf{nf}$$

where in the core the transport-neutral case does not arise (all core type values are canonical, Lemma F.1), path application $\mathsf{ne}\,r$ is restricted to r not an endpoint (the η/boundary rules fire otherwise), and hcomp-neutrals occur only at $\mathbb{N}$ over a neutral base with an undecided system (Lemma F.22). Read-back targets exactly this grammar, clause by clause of quote (Appendix E).

F.2. **Worlds: depths, dimensions, and faces.** Contexts of the calculus contain term variables, *interval variables* and *cofibration restrictions* Γ, φ; and the boundary conditions of hcomp hold only *under faces*. The logical relation must therefore be indexed not by depths alone but by:

Definition F.3 (Worlds)**.** Worlds are defined relative to a fixed *sorted context shape*: the context determines, for each level $< d$, whether it is a term level or an interval level, and all notions below respect this sorting (the implementation records it in the context; values never confuse the sorts, by the typing of the smart constructors). A *world* is then a pair $w = (d,\psi)$ of a depth d (making available the term and interval levels $< d$, as sorted by the context shape) and a cofibration ψ over the interval levels $< d$ (the conjunction of the faces currently assumed; $\psi = \mathsf{i1}$ is the empty assumption, $\psi = \mathsf{i0}$ the absurd one). A *world map* $(d',\psi') \to (d,\psi)$ is a dimension substitution ξ from the interval levels $< d$ to $\mathrm{DM}(< d')$ (extended by the identity on term levels, with $d \le d'$ on the term part) such that $\psi' \models \psi\langle\xi\rangle$. Weakenings ($d \le d'$, $\xi = \mathrm{id}$, face strengthening $\psi' \models \psi$) and face restrictions ($\psi' := \psi \wedge \varphi$) are the two generating cases used below.

Definition F.4 (Semantic context satisfaction)**.** Related environments at a context, at a world $w = (d,\psi)$: term variables pointwise related (at the types of the context, each in the

preceding environments, at w); interval variables assigned *the same* interval value over the levels $< d$ on both sides (Definition 5.1's literal-interval discipline); and for a restriction entry φ of the context, $\psi \models \varphi[\rho]$ — the world must satisfy the restriction.

At the absurd world ($\psi = \mathsf{i0}$, i.e. $\psi \models \bot$) all values of all types are related by convention; this is what makes empty-intersection compatibility conditions (such as (P3)) genuinely vacuous rather than informally so.

F.3. **The logical relation.** For each syntactic type T, world $w = (d, \psi)$, and pair of environments ρ, ρ' related at T's context at w (Definition F.4), a relation $v \sim^{w}_{T[\rho,\rho']} v'$ on values, by structural recursion on T; every clause is implicitly *Kripke over worlds*: it quantifies over all $w' \to w$, with the values transported by the corresponding dimension action, and all clauses are Kleene (the mentioned applications and instantiations are required to be defined):

- $\mathbb{N}$: both force to the same numeral, or to neutrals with equal read-back (including stuck hcomp-neutrals over neutral bases);
- $\Pi x{:}T.S$: for all worlds $w' \to w$ and all $u \sim^{w'}_{T[\rho,\rho']} u'$: $f\,u \sim^{w'}_{S[\rho+[u],\,\rho'+[u']]} f'\,u'$;
- $\Sigma x{:}T.S$: $\mathsf{fst}u \sim^{w}_{T[\rho,\rho']} \mathsf{fst}u'$ and $\mathsf{snd}u \sim^{w}_{S[\rho+[\mathsf{fst}u],\,\rho'+[\mathsf{fst}u']]} \mathsf{snd}u'$;
- $\mathsf{PathP}\,(\langle i\rangle T)\,l\,r$: for every world $w' \to w$ and interval value ι over w': $p\,\iota \sim^{w'}_{T[\rho+[\iota],\,\rho'+[\iota]]} p'\,\iota$; at $\iota = \mathsf{i0}$ (resp. $\mathsf{i1}$) the boundary rules force the values of l (resp. r), so the endpoint conditions are instances of the same clause.

A *computable type in ρ at w* is one all of whose stored components are so related (for PathP: the line, at every instantiation over every future world, and both endpoints).

Definition F.5 (Related systems)**.** Systems $[\varphi_k \mapsto t_k]$, $[\varphi_k \mapsto t'_k]$ (same cofibrations, by Definition 5.1) are related at T, $w = (d, \psi)$ if: for each k, the tubes are related *at the restricted world* $(d, \psi \wedge \varphi_k)$, at every instantiation of their interval binder; and for each pair k, l, the tubes agree (are related to each other) at $(d, \psi \wedge \varphi_k \wedge \varphi_l)$. A base u_0 is *adapted* to the system if $u_0 \sim^{(d,\psi\wedge\varphi_k)} t_k(\mathsf{i0})$ for each k — the semantic boundary condition.

Lemma F.6 (PER laws, environment irrelevance, world functoriality)**.** *Each $\sim^{w}_{T}$ is symmetric and transitive; replacing ρ, ρ' by environments related to them yields the same relation; and relatedness is* functorial in the world*: for every world map $\theta : w' \to w$ with dimension action $\langle\theta\rangle$, $v \sim^{w}_{T} v' \implies v\langle\theta\rangle \sim^{w'}_{T} v'\langle\theta\rangle$ (*face restriction / weakening lemma*).*

Proof. Structural induction on T. PER laws: $\mathbb{N}$ by equality of numerals and read-backs; Π by swapping and composing through a common argument; Σ componentwise with environment irrelevance at S; PathP pointwise. Environment irrelevance is inherited from the clauses' quantification. Functoriality: every clause quantifies over all future worlds, and world maps compose, so the quantified conditions restrict; the only content is at the base cases — the dimension action commutes with forcing (a face decided at w stays decided at w' since $\psi' \models \psi\langle\theta\rangle$; new decisions can only collapse both sides identically, L5), and read-back commutes with the action (Lemma L14) — so numeral/neutral disjuncts are preserved. At the absurd world the convention closes every case. □

Lemma F.7 (Boundary preservation)**.** *Let the system and adapted base be related at T, $w = (d, \psi)$ (Definition F.5), and suppose the* hcomp *at T is defined (as it is in each case of Lemmas F.22–F.23). Then at each restricted world $(d, \psi \wedge \varphi_k)$, the composite is related to the tube top $t_k(\mathsf{i1})$.*

Proof. At $(d, \psi \wedge \varphi_k)$ the face φ_k is satisfied, so forcing decides it and the composite collapses to $t_k(\mathsf{i1})$ on both sides (the hcomp face rule; L5); the tube tops are related there by Definition F.5. Functoriality (Lemma F.6) transports the ambient relatedness data to the restricted world, so the collapse happens in a coherent context. □

F.4. **Reflect and reify, former by former.** Reflect and reify are proved by *one* mutual structural induction on the syntactic type: the lemma for a former may use both lemmas at that former's proper subexpressions. We state them per former, as separate lemmas, so that each obligation is inspectable in isolation; the induction discipline is: L(T) may invoke L′(T′) for any T' a proper subterm of T and any L′ among the eight lemmas below.

Lemma F.8 (Reflect at $\mathbb{N}$)**.** *Neutrals n, n' of type $\mathbb{N}$ with $\mathsf{quote}(n) = \mathsf{quote}(n')$ satisfy $n \sim_{\mathbb{N}} n'$.*

Proof. The neutral disjunct of the $\mathbb{N}$-clause, verbatim. □

Lemma F.9 (Reify at $\mathbb{N}$)**.** $v \sim_{\mathbb{N}} v' \implies \mathsf{quote}(v) = \mathsf{quote}(v')$*, both defined.*

Proof. Same-numeral case: numerals quote to themselves. Neutral case: the clause *is* read-back equality. □

Lemma F.10 (Reflect at Π)**.** *Let $\Pi x{:}T.S$ be computable in ρ, ρ' and n, n' neutrals of that type with equal read-back. Then $n \sim_{\Pi x:T.S\,[\rho,\rho']} n'$.*

Proof. Given $d' \geq d$ and $w \sim_T w'$: by reify at the subterm T (the mutual-induction instance of this lemma family), $\mathsf{quote}(w) = \mathsf{quote}(w')$, so the applications $n\,w$, $n'\,w'$ are neutrals with equal read-back $\mathsf{quote}(n)\,\mathsf{quote}(w) = \mathsf{quote}(n')\,\mathsf{quote}(w')$; reflect at the subterm S (in the extended, related environments) relates them at S, which is the Π-clause's requirement. Here and below, "reflect/reify at T" always abbreviates the mutual-induction instance at the proper subterm T. □

Lemma F.11 (Reify at Π)**.** $f \sim_{\Pi x:T.S\,[\rho,\rho']} f' \implies \mathsf{quote}(f) = \mathsf{quote}(f')$*, both defined.*

Proof. Let x_d be the generic neutral of type T at the current depth; it is related to itself by reflect at T (its read-back is the variable). The Π-clause gives $f\,x_d \sim_{S[\rho+[x_d]]} f'\,x_d$, defined; reify at S gives equal body read-backs; quote at Π-type values produces the λ-abstraction of exactly that body (Appendix E), so the read-backs agree. □

Lemma F.12 (Reflect at Σ)**.** *Under the same hypotheses at $\Sigma x{:}T.S$: $n \sim_{\Sigma x:T.S} n'$.*

Proof. $\mathsf{fst}n, \mathsf{fst}n'$ are neutrals with equal read-back, related at T by reflect at T; hence the second-component type $S[\rho{+}[\mathsf{fst}n]]$ is computable (Lemma F.6, environment irrelevance), and $\mathsf{snd}n, \mathsf{snd}n'$ are neutrals with equal read-back, related at it by reflect at S. Both components together are the Σ-clause. □

Lemma F.13 (Reify at Σ)**.** $u \sim_{\Sigma x:T.S} u' \implies \mathsf{quote}(u) = \mathsf{quote}(u')$.

Proof. Componentwise by reify at T and at S (the latter in the environments extended by the related first components); quote produces the pair of the component read-backs. □

Lemma F.14 (Reflect at PathP)**.** *Let $P := \mathsf{PathP}(\langle i\rangle T)\,l\,r$ be computable in ρ, ρ' and n, n' neutrals of type P with equal read-back. Then $n \sim_P n'$.*

Proof. Take any interval value ι. If ι is not forced to an endpoint, $n\,\iota$, $n'\,\iota$ are neutral path applications; their read-backs agree because interval read-back is structural and the spine read-backs agree; reflect at T (in the environments extended by ι) relates them. If $\iota = \mathsf{i0}$ (resp. $\mathsf{i1}$), the boundary rule returns the stored endpoint values of l (resp. r) on both sides — related because P is computable (its endpoint components are related by definition of computable type). Both cases are what the PathP-clause demands. □

Lemma F.15 (Reify at PathP)**.** $p \sim_P p' \implies \mathsf{quote}(p) = \mathsf{quote}(p')$.

Proof. Instantiate at the generic interval level ι_d: related bodies at $T[\rho{+}[\iota_d]]$ by the clause; reify at T gives equal body read-backs; interval arguments of any neutral path applications occurring inside compare equal by canonical antichain-DNF (sound and complete for the free De Morgan algebra); quote produces the path abstraction of the common body. □

The mutual induction across the eight lemmas is well-founded by Remark F.2: every cross-reference is at a proper syntactic subterm.

F.5. **Conversion correctness, restructured.** Conversion correctness splits into a purely *syntactic* characterization and two *type-level* semantic lemmas. The split is deliberate: read-back equality and quote–eval idempotence are different theorems, and the load-bearing semantic content is needed *only at type values*, where the core's type-value inversion (Lemma F.1) makes it provable by structural induction on syntax — the generic-to-arbitrary passage never has to be performed on arbitrary function *values*.

Lemma F.16 (Conversion is read-back equality)**.** *For values of a core type,* $\mathsf{conv}(v, v', T\text{-}value)$ *accepts iff* $\mathsf{quote}(v) = \mathsf{quote}(v')$*, and the accepting run is defined iff both read-backs are.*

Proof. Purely syntactic, no logical relation involved. ($\Rightarrow$) Induction on the accepting run: each conv clause compares exactly what the corresponding quote clause emits — the η-directed $\Pi/\Sigma/\mathsf{PathP}$ clauses instantiate at the same generic levels as quote's binder cases; numerals, neutral spines and interval arguments (DNF) coincide with their quotations. ($\Leftarrow$) Induction on the common read-back along the normal-form grammar of Section F.1: each production is matched by exactly one conv clause. Definedness transfers as in Theorem 6.3: the two traversals visit the same instantiations. □

Lemma F.16 already yields, with no semantic input, that $\equiv_{\mathrm{alg}}$ restricted to the core is an equivalence relation and a congruence in all value-forming positions: it is equality of read-backs.

Lemma F.17 (Type-level idempotence)**.** *Let* $F = \mathsf{eval}(T, \rho)$ *be a computable type value at* w*. Then* $\mathsf{quote}(F)$ *is a syntactic core type, it is well-typed in the generic context of* w*, and* $\mathsf{eval}(\mathsf{quote}(F), \rho_{\mathrm{gen}})$ *is defined and determines* the same relation *as* F*:* $\sim_{\mathsf{quote}(F)[\rho_{\mathrm{gen}}]} = \sim_{T[\rho]}$.

Proof. Structural induction on T, using type-value inversion throughout. $\mathbb{N}$: $\mathsf{quote}(F) = \mathbb{N}$, evaluation returns $\mathsf{v}\mathbb{N}$, same clause. $\Pi x{:}T.S$: $\mathsf{quote}(F) = \Pi x{:}\mathsf{quote}(D).\mathsf{quote}(C\,x_d)$ where D is the domain value and $C\,x_d$ the codomain instantiated at the generic level; by the induction hypothesis at T the quoted domain re-evaluates to a type value with the same relation, and for every $u \sim u'$ at a future world the quoted codomain re-evaluates over $\rho_{\mathrm{gen}}{+}[u]$ — by Lemma F.1 both the original and the re-evaluated codomain instantiations are evaluations of the *syntactic* S (resp. its read-back) in related environments, so the induction hypothesis at S applies pointwise. Σ, PathP: the same pattern (for PathP, pointwise over interval

values at all future worlds, endpoints by the induction hypothesis at the endpoint type). Well-typedness of the read-back is the same induction read syntactically. □

Lemma F.18 (Reindexing at PathP)**.** *Let L, L' be line closures of core type (over the syntactic types T, T', by inversion) and suppose* $\mathsf{eval}(T, \rho{+}[\iota_d])$ *and* $\mathsf{eval}(T', \rho'{+}[\iota_d])$ *determine the same relation at the generic interval level ι_d (at all future worlds). Then for* every *interval value ι over any future world,* $\mathsf{eval}(T, \rho{+}[\iota])$ *and* $\mathsf{eval}(T', \rho'{+}[\iota])$ *determine the same relation.*

Proof. World functoriality (Lemma F.6): the map that sends the generic level to ι is a world map (Definition F.3), the dimension action transports the generic instance to the ι-instance, and evaluation commutes with the action on the interval entry of the environment (the action acts pointwise on environments, Section 5.1). Note this is a *dimension* substitution — available as a world map — not a term substitution; no appeal to Lemma 7.3 is needed. □

Lemma F.19 (Type conversion soundness)**.** *Let F, F' be computable type values at w. If* $\mathsf{conv}(F, F', \mathcal{U})$ *accepts, then F and F' determine the same relation.*

Proof. By Lemma F.16, $\mathsf{quote}(F) = \mathsf{quote}(F')$; by Lemma F.17 both F and F' determine the same relation as the evaluation of that common syntactic type in the generic environment. (Unfolded, the generic-to-arbitrary passage happens inside Lemma F.17's Π case by structural induction on the *syntactic* codomain, and at PathP by Lemma F.18 — never on arbitrary function values.) □

Lemma F.20 (Conversion completeness)**.** *If $v \sim_{T[\rho,\rho']} v'$ then* $\mathsf{conv}(v, v', T\text{-}value)$ *accepts.*

Proof. By the reify lemmas (F.9, F.11, F.13, F.15) the read-backs are equal; Lemma F.16 (⇐) concludes. □

Remark F.21 (What happened to term-level semantic soundness)**.** The development never needs "conv accepts ⇒ related" for arbitrary function *values*: the fundamental theorem's conversion case switches *type* predicates (Lemma F.19); the admissibility package uses the equivalence/congruence structure of $\equiv_{\mathrm{alg}}$, which Lemma F.16 provides syntactically; and the constancy argument uses idempotence at *type* values (Lemma F.17). We therefore do not claim the term-level statement, and no proof in this paper depends on it. This is the precise resolution of the apparent generic-to-arbitrary gap: at types it is closed by structural induction on syntax; at terms it is never used.

F.6. hcomp **computability, former by former.** *Induction discipline for the Kan lemmas.* Lemmas F.22–F.24 are proved by *one* simultaneous structural induction on the syntactic type: the Σ-case of composition invokes transport at the subterm S, and the PathP-case of transport invokes composition at the subterm T — every cross-reference between the two lemmas is at a proper syntactic subterm, so the joint induction is well-founded by Remark F.2. We keep the statements separate for readability.

Lemma F.22 (hcomp at $\mathbb{N}$)**.** *If the system is related and the base adapted, at w (Definition F.5), the* hcomp*s at $\mathbb{N}$ are defined and related at w, with the boundary behaviour of Lemma F.7.*

Proof. If some face is decided true, both sides return that tube's top (L5), related by assumption. Otherwise, by cases on the (common, by the $\mathbb{N}$-clause) shape of the base.

Numeral base: by induction on the numeral, using the kernel's computation rules — an hcomp at $\mathbb{N}$ with base ze (resp. $\mathsf{su}\,m$) peels the tubes through the constructor (ze/su commute with hcomp): tube tops over a numeral base at undecided faces are themselves related numeral-shaped values by the boundary condition, and the peeled composite recurses at m. *Neutral base:* both sides form the stuck hcomp-*neutral* with equal read-back (tubes reified by the reify lemmas under their interval binder; equal cofibrations), covered by the neutral part of the $\mathbb{N}$-clause. *Closed instance remark* (used for canonicity): a closed cofibration contains no interval variables, hence is decided; so closed hcomps at $\mathbb{N}$ always take the decided-face or numeral path and compute to numerals — no closed stuck composite exists. □

Lemma F.23 (hcomp at Π, Σ, PathP)**.** *Same hypotheses (Definition F.5, at w); the structural composites are defined and related at w, with the boundary behaviour of Lemma F.7.*

Proof. Π*:* the composite is the closure value $\lambda w.\,\mathsf{hcomp}\,[\vec{\varphi} \mapsto \vec{t}\,w]\,(u_0\,w)$; applying to related $w \sim_T w'$ gives related tubes and base at $S[\rho{+}[w]]$ (the Π-clause, pointwise), and the induction hypothesis of the outer structural recursion on T (the composite at S) closes the case. Σ*:* let the input be $\mathsf{hcomp}^{\,\iota}\,(\Sigma x{:}T.S)\,[\vec{\varphi}_k \mapsto t_k]\,u_0$ in environment ρ. The first components compose at T:

$$c \;:=\; \mathsf{hcomp}^{\,\iota}\,T\,[\vec{\varphi}_k \mapsto \mathsf{fst}t_k]\,(\mathsf{fst}u_0) \;:\; T[\rho],$$

and the kernel forms their *filler*, the truncation of the same composition at ι:

$$\overline{c}(\iota) \;:=\; \mathsf{hcomp}^{\,\iota'}\,T\,[\vec{\varphi}_k \mapsto \mathsf{fst}t_k(\iota \wedge \iota'),\ (\iota{=}0) \mapsto \mathsf{fst}u_0]\,(\mathsf{fst}u_0) \;:\; T[\rho],$$

with $\overline{c}(0) = \mathsf{fst}u_0$ and $\overline{c}(1) = c$, related to its primed counterpart at every ι by this lemma at T (the truncated system is again a related system). The second components then compose *heterogeneously along the line* $\iota \mapsto S[\rho{+}[\overline{c}(\iota)]]$:

$$\mathsf{snd}(\text{result}) \;:=\; \mathsf{comp}^{\,\iota}\,\big(S[\overline{c}(\iota)/x]\big)\,[\vec{\varphi}_k \mapsto \mathsf{snd}t_k]\,(\mathsf{snd}u_0) \;:\; S[\rho{+}[c]],$$

whose transport part is along instantiations of the proper subexpression S at the related environments $\rho{+}[\overline{c}(\iota)]$ (Lemma F.24 at S), and whose hcomp part is the induction hypothesis at S. The boundary premises are inherited: on φ_k, the pair t_k has $\mathsf{fst}t_k$ matching the filler on that face ($\overline{c}(\iota) = \mathsf{fst}t_k(\iota)$ there, by the filler's face rule), so $\mathsf{snd}t_k$ is a legitimate tube for the heterogeneous composite; at $\iota = 0$ the composite starts at $\mathsf{snd}u_0 : S[\rho{+}[\mathsf{fst}u_0]] = S[\rho{+}[\overline{c}(0)]]$. The resulting pair is related at $\Sigma x{:}T.S$ by the Σ-clause, whose second-component environment is $\rho{+}[c]$, exactly the filler at 1. All instantiations are of proper subexpressions (Remark F.2). PathP*:* the composite at $\mathsf{PathP}(\langle i \rangle T)\,l\,r$ is $\langle j \rangle\,\mathsf{hcomp}\,[\vec{\varphi} \mapsto \vec{t}\,j,\ j{=}0 \mapsto l,\ j{=}1 \mapsto r]\,(u_0\,j)$: the system is *extended with the endpoint tubes.* Endpoint coherence — the reviewer-facing detail — is exactly the boundary check: at $j = \mathsf{i0}$ the added face is decided true and the composite collapses to l (the hcomp face rule), matching the PathP-clause's endpoint condition; likewise at $\mathsf{i1}$; at generic j the composite is the induction hypothesis at T with the enlarged (still pairwise-compatible: the original tubes agree with l, r at the endpoints by *their* boundary conditions) system. □

F.7. **Prioritized transport computability, former by former.**

Lemma F.24. *If the family closure (over a syntactic core type in one interval binder) and the argument are related, the prioritized transport is defined and related at the endpoint type.*

Proof. Positive branch: Theorem 6.2; its ingredients — read-back/action commutation (Lemma L14) and type-level idempotence — are available for the core as Lemma F.17. *Negative branch*, by cases on the family's head (Lemma F.1; there is no Glue case and no neutral case):

$\mathbb{N}$-*line:* the family value at the generic dimension has head $\mathsf{v}\mathbb{N}$ with no arguments, so it is ℓ-free and the check fired (Theorem 6.3); this branch is unreachable.

Π-*line* $\langle i\rangle\Pi x{:}T.S$: the contractum is $\lambda x_1.\,\mathsf{transp}^i S[w\,x_1\,i/x]\,(f\,(w\,x_1\,0))$ with $w\,x_1\,i = \mathsf{transp}^j T(i \vee \neg j)\,x_1$. Apply to related $x_1 \sim x_1'$ at the endpoint domain: $w\,x_1\,i$ is a transport along the *domain line* — the syntactic subexpression T under a reparametrized interval environment — so this lemma applies at T (structural recursion, Remark F.2), giving related fills, in particular related $w\,x_1\,0$ at the source domain; f applied to them is related at $S[\rho{+}[w\,x_1\,0]]$ by the Π-clause; the outer transport is along $\langle i\rangle S[w\,x_1\,i/x]$ — the subexpression S in the environment extended by the fill — so this lemma at S closes the case, and the results assemble by the Π-clause at the endpoint type.

Σ-*line* $\langle i\rangle\Sigma x{:}T.S$: the first component transports along T (this lemma at T). The *codomain typing detail*: the second component transports along $\langle i\rangle S[\overline{u}\,i/x]$ where $\overline{u}\,i = \mathsf{transp}^j T(i \wedge j)\,(\mathsf{fst}u)$ is the first component's fill; by this lemma at T the fills are related at $T[\rho{+}[\iota]]$ for each ι, so the environments $\rho{+}[\overline{u}\,i]$ are related, the line $\langle i\rangle S[\overline{u}\,i/x]$ is a computable family (its instantiations are evaluations of the subexpression S in related environments), and this lemma at S applies. The pair is related at the endpoint type by the Σ-clause, whose second-component environment $\rho{+}[\mathsf{transp}\,T\,(\mathsf{fst}u)]$ is exactly the fill at $\mathsf{i1}$.

PathP-*line* $\langle i\rangle\mathsf{PathP}(\langle j\rangle T)\,l\,r$ (where T, l, r may mention i): the contractum and its obligations, in derivation form. The structural rule produces

$$\langle j\rangle\,\mathsf{comp}^i\,T(i,j)\,[\,j{=}0 \mapsto l(i),\;\; j{=}1 \mapsto r(i)\,]\,(p\,j),$$

whose premises are exactly the composition rule's:

$$j{=}0: \quad l(0) \equiv p\,0 \qquad \text{the source typing of } p \tag{P1}$$

$$j{=}1: \quad r(0) \equiv p\,1 \qquad \text{likewise} \tag{P2}$$

$$j{=}0 \wedge j{=}1: \quad \text{vacuous} \qquad (0 = 1) = \bot \tag{P3}$$

— (P1)/(P2) hold because $p : \mathsf{PathP}(\langle j\rangle T(0,j))\,l(0)\,r(0)$ and path application at an endpoint returns that endpoint. The composite unfolds as transport of the base and tubes along $\langle i\rangle T(i,j)$ followed by an hcomp at $T(1,j)$ with the transported endpoint tubes; each transport is this lemma at the subterm T, and the hcomp is Lemma F.23 at T. The *endpoint coherence* of the result — required by the PathP-clause at the target type $\mathsf{PathP}(\langle j\rangle T(1,j))\,l(1)\,r(1)$ — is the face computation:

$$\begin{aligned}(\text{result})\,\mathsf{i0} &= \mathsf{comp}\text{-value on the decided face } j{=}0 = \text{top of the } l\text{-tube}\\ &= \text{the } \mathsf{transp}^i\,T(i,0)\text{-corrected } l \text{ at } i{=}1 \;\equiv\; l(1),\end{aligned} \tag{E0}$$

$$(\text{result})\,\mathsf{i1} \equiv r(1) \quad \text{symmetrically}, \tag{E1}$$

where the last step of (E0) is the composition rule's face collapse (hcomp on a decided face returns the tube top) together with the tube's own transport landing at $l(1)$ — precisely the boundary rule of comp. So the contractum inhabits the target path type with the required endpoints, and relatedness at generic j is the two ingredients above. □

F.8. **The fundamental theorem, and the corollaries.**

Theorem F.25. *Every rule of* $T^{\mathrm{core}}_{\mathrm{alg}}$ *preserves semantic typing (Definition 7.5, with* $\equiv^{\mathrm{val}}_{\mathsf{eval}(T,\rho)}$ *read as* $\sim_{T[\rho,\rho']}$*).*

Proof. Induction on the typing derivation. Variables: environment relatedness. λ/application, pairing/projections, path abstraction/application: the matching clauses of Section F.3, with definedness carried by each clause. $\mathbb{N}$-introductions: numerals. natrec: by the $\mathbb{N}$-clause the scrutinee is a numeral (compute by an inner induction on the numeral, the step case using the Π-clause of the step premise) or a neutral (both sides form recursor neutrals with equal read-back — motive by reify under its binder — related by reflect at the motive instance). transp: Lemma F.24. hcomp: Lemmas F.22 and F.23. Conversion rule: Lemma F.19 (the switch of type predicates), with Lemma F.20 and Lemma F.6 (environment/type irrelevance). □

Corollary F.26. *For the minimal core, unconditionally: evaluation, read-back and conversion terminate on well-typed inputs; canonicity (closed naturals are numerals: the* $\mathbb{N}$*-clause, no closed neutrals, and no closed stuck composites by the closed-instance remark of Lemma F.22); consistency; and the admissibility package of Theorem 7.8.*

F.9. **A paper proof of the path separation.** The read-back characterization of conversion (Lemma F.16) upgrades the machine witness of Lemma 4.5 to a paper theorem, for every fixed core type.

Lemma F.27 (hcomp/spine separation)**.** *Let* T *be a core type over a core context,* n *a neutral of type* T*, and* $H := \mathsf{hcomp}^{\iota}\, T\, [\vec{\varphi} \mapsto \vec{t}\,]\, (n)$ *a composite whose faces* $\vec{\varphi}$ *are undecided at the current world and whose tubes are neutral-based values constant in the fill dimension* ι*. Then* $\mathsf{quote}(H) \neq \mathsf{quote}(n)$*; hence, by Lemma F.16,* $H \not\equiv_{\mathrm{alg}} n$*.*

Proof. Structural induction on T, following the kernel's structural hcomp rules (which are also how quote sees the values). $\mathbb{N}$*:* the base is neutral, so H is a stuck hcomp-neutral (Section F.1); its read-back is an hcomp-term, while $\mathsf{quote}(n)$ is an elimination spine — distinct productions of the normal-form grammar. $\Pi x{:}T_1.S$*:* read-back η-expands both sides at the generic x_d; on the left, the structural rule turns $H\, x_d$ into an hcomp at $S[x_d]$ whose base $n\, x_d$ is again a neutral spine and whose tubes remain fill-constant and neutral-based; on the right, $n\, x_d$ is a spine. The induction hypothesis at S gives distinct body read-backs, hence distinct λ-bodies. $\Sigma x{:}T_1.S$*:* read-back compares first projections; $\mathsf{fst}H$ is the composite of the first components over the spine $\mathsf{fst}n$ (fill-constant, neutral-based), and the induction hypothesis at T_1 separates already the first components. $\mathsf{PathP}(\langle k\rangle T_1)\, l\, r$*:* read-back instantiates at the generic level k_d; the structural rule extends the system with the endpoint tubes $k{=}0 \mapsto l$, $k{=}1 \mapsto r$ — undecided at generic k_d, so the composite does not collapse — with base $n\, k_d$, a spine; the induction hypothesis at T_1 applies with the enlarged (still fill-constant, neutral-based) system. In every case the faces stay undecided at the generic instantiations, so no collapse intervenes, and the separation bottoms out at $\mathbb{N}$ in distinct grammar productions. □

Proposition F.28 (Path separation in the core, on paper)**.** *For every core type* A *and the generic context* $(a\, b : A,\ p : \mathsf{Path}\, A\, a\, b)$*:*

$$\langle j\rangle\, \mathsf{hcomp}^{\iota} A\, [\, j{=}0 \mapsto a,\ j{=}1 \mapsto b\,]\, (p\, j)\ \not\equiv_{\mathrm{alg}}\ p,$$

unconditionally.

Proof. Conversion at the path type instantiates both sides at the generic level j_d: the faces $j_d{=}0$, $j_d{=}1$ are undecided, the base $p\,j_d$ is a neutral spine, and the tubes a, b (values of the context variables) are generic neutrals, constant in the fill dimension. Lemma F.27 at A separates the two sides; Lemma F.16 turns the read-back difference into rejection. Every step lives in the minimal core, so Corollary F.26 makes the statement unconditional. Consequently the no-go analysis of Section 4.3 — both the derivation of (†) in T_{eq} and its refutation by the algorithm — is established on paper within the unconditional layer; the machine probes witness, in addition, the full implemented calculus with A a universe variable. □

F.10. **The derived J, explicitly.**

Proposition F.29 (Strict J in the minimal core; a schema)**.** *Work in the core over a context* $\Gamma = (x : A$; *and, in the extended context* $y : A,\ p : \mathsf{Path}\, A\, x\, y,$ *a core type* C*), with* $d : C[x, \mathsf{refl}_x]$*. Define, as usual in cubical type theory,*

$$\mathsf{J}_C\, d\, y\, p \;:=\; \mathsf{transp}^{i}\, C\big[\, p(i),\ \langle j\rangle\, p(i \wedge j)\,\big]\ \mathsf{i0}\ d.$$

Then: (i) J_C *is well-typed in the core (the family is a core type in the interval binder* i*, using the connection* $i \wedge j$*; no* Glue*, universe or HIT is involved); (ii)* $\mathsf{J}_C\, d\, x\, \mathsf{refl}_x \equiv_{\mathrm{alg}} d$*, unconditionally.*

Proof. (i) is the standard typing of the connection square: at $i = 0$ the family is $C[p(0), \langle j\rangle p(0)] = C[x, \mathsf{refl}_x]$ by the boundary and connection laws ($0 \wedge j = 0$), so d is accepted at the source. (ii) Substitute $p := \mathsf{refl}_x$ and evaluate the transport's family at the fresh dimension ℓ: $p(\ell)$ evaluates to (the value of) x, and $\langle j\rangle\, p(\ell \wedge j)$ evaluates to the constant path abstraction over x — the connection is absorbed by the DNF normalization ($\ell \wedge j$ instantiated on the constant path collapses; machine witness `strictConnZeroD`). So the family value is the value of $C[x, \mathsf{refl}_x]$ in an environment not involving ℓ, and by Lemma L1 its read-back does not contain ℓ. By Theorem 6.3 the check fires; the positive branch returns the value of d; by Lemma F.20 the algorithmic equality $\mathsf{J}_C\, d\, x\, \mathsf{refl}_x \equiv_{\mathrm{alg}} d$ holds. Every ingredient — evaluation, check, conversion — is inside the minimal core, so Corollary F.26 applies and no assumption is used. Machine witness: `strictJReflD`. □

Remark F.30 (Schema, not internal term)**.** The core has no universe, so J_C is a *meta-level schema*: one derived eliminator per core type C in the extended context, not a single internal universe-polymorphic term. This is the usual status of J in a universe-free fragment and involves no loss: Proposition F.29 is proved uniformly in C. (In the full calculus, where universes exist, the internal J is the same construction at a universe-valued motive; it then lives in the relative layer.)

F.11. **The S^1 extension.**

Proposition F.31 (S^1, proof sketch)**.** *The results of this appendix extend to the fragment enlarged by* S^1 *(*base*,* loop*, its eliminator, and* hcomp*-cells).*

Proof sketch (deliberately so labelled). The S^1-clause of the relation adds: both base; or loop at DNF-equal interval values; or componentwise-related hcomp-cells with equal cofibrations; or neutrals with equal read-back. Transport along a constant S^1-line always takes the

positive branch (as for $\mathbb{N}$). The eliminator computes on base and loop by its two rules, acts on an hcomp-cell by composing in the motive fiber against the tube-transformed system (the kernel's tube-transformer closure), and is neutral on neutrals; boundary computation ($\mathsf{loop}(\mathsf{i0}) = \mathsf{loop}(\mathsf{i1}) = \mathsf{base}$) is the face collapse. Filling in the eliminator-over-hcomp case at the level of detail of Sections F.6–F.7 is routine but lengthy, and we deliberately keep it *outside* the unconditional theorem; the artifact provides machine evidence (the S^1 probes, the winding-number development $\pi_1(S^1) \cong \mathbb{Z}$, and the switchover probes for higher-inductive constructors). □

What the core deliberately omits. Glue (hence ua and transpGlue), universes (hence large elimination), and all HITs (with S^1 recovered as Proposition F.31). These are exactly the ingredients whose base metatheory is the content of [4, 5]; extending the discharge to them is the remaining gap between Theorem A′ and Theorems A/B, and we prefer to leave it visibly open rather than to compress those proofs beyond inspectability.

Appendix G. The Sterling–Angiuli bridge

This appendix makes precise which parts of the base component (B) are discharged by the normalization theorem of Sterling–Angiuli [5] for the Cartesian layer (Theorem A), and which parts remain a transposition obligation. It is based on the published LICS 2021 paper (arXiv:2101.11479v2).

G.1. **What their theorem provides.** Their subject is intensional *Cartesian* cubical type theory, presented as the locally Cartesian closed category of judgments generated by an explicit signature: connectives Π, Σ, path, glue (with Equiv defined without universes), the circle S1 with its eliminator, and Kan operations hcom and

$$\mathsf{coe} : \textstyle\prod_{A:\mathbb{I}\to\mathsf{tp}} \prod_{r,s:\mathbb{I}} \mathsf{tm}(A(r)) \to \{\mathsf{tm}(A(s)) \mid r = s \hookrightarrow a\},$$

with cofibration classifier $\mathbb{F}$ closed under $\wedge, \vee, \forall_{\mathbb{I}}, =_{\mathbb{I}}$. The method is synthetic Tait computability — a gluing model over the generic model, with the novel device of *stabilized neutrals* (neutrals indexed by the cofibration recording where they cease to be neutral); it is, by design, *not* evaluation-based. The main results are: the normalization theorem (their Theorem 42), a bijection between equivalence classes of terms in context and β/η-normal forms; injectivity of type constructors (their Theorem 43); and an algorithm deciding judgmental equality (their Corollary 47). Two scope restrictions are explicit in their paper: *universes are not treated* (they note the expected extension via cumulative universes and the tininess of the interval, but do not carry it out), and the only higher inductive type is the circle. For the De Morgan variant they state that the argument "carries over mutatis mutandis"; the adaptation is likewise not carried out.

G.2. **Item-by-item correspondence.**

component (B) item	status under [5]
normal-form language; uniqueness; inconvertibility of distinct normal forms (L13, second half)	**provided**: Theorem 42 (bijection with β/η-normal forms), for $\Pi, \Sigma, \mathsf{path}, \mathsf{glue}, \mathsf{S1}$, no universes
injectivity of type constructors	**provided**: Theorem 43
decidability of the base judgmental equality	**provided**: Corollary 47
predicate clauses of $\equiv^{\mathrm{val}}$ for their connectives	**provided in substance**: their computability structures (with stabilized neutrals) are the proof-relevant counterpart of Definition 5.3's clauses; the transposition to a concrete Kripke PER over our domain is mechanical but must be written
environment-form fundamental cases for a concrete NbE evaluator (Appendix D, (B)-cases); read-back correctness for that evaluator (L13, first half); evaluator termination content	**not literal**: their normalization function arises from the glued model, not from an operational evaluator; the standard NbE-correctness argument for a concrete evaluator against their normal forms is a known pattern but is not in their paper
universes ($\equiv^{\mathrm{val}}_{\mathcal{U}}$, universe rules of Appendix D)	**gap**: outside their theorem; expected but unproven extension
HITs beyond the circle (Section 4.2)	**gap**: outside their theorem
De Morgan instance	**gap**: "mutatis mutandis" claim, not carried out — which is why our De Morgan layer is Theorem B, relative to (B)

G.3. **Primitive translation: boundary-conditioned coe vs. φ-annotated transp.** Their coercion is regular *on the diagonal*: the codomain $\{\mathsf{tm}(A(s)) \mid r = s \hookrightarrow a\}$ builds $\mathsf{coe}(A, r, r, a) = a$ into the *type* of the primitive — the Cartesian counterpart of our ($\mathrm{S_{i1}}$), expressed as a boundary condition rather than a formula annotation. Our $\mathsf{transp}^i A\, \varphi\, u$ translates as $\mathsf{coe}(\lambda i.A, 0, 1, u)$, with the constancy formula φ corresponding to the cofibration under which the line is required constant; rule (R) then reads: coerce along a line whose *normal form* is degenerate as if $r = s$. The switchover analysis of Section 4 is insensitive to the difference: the derivation (4.1)–(†) uses only the diagonal rule, the structural path rule and congruence, all of which the Cartesian signature has, and the residue is an hcom with fill-constant tubes.

G.4. **The precise statement of Theorem A.**

> **Theorem A.** Over the fragment $\Pi, \Sigma, \mathsf{path}, \mathsf{glue}, \mathsf{S1}$ of Cartesian cubical type theory — the fragment covered by [5] — the results of Sections 7–9 (admissibility, termination, canonicity, definitional $\mathsf{J}\, d\, \mathsf{refl}$) hold for the prioritized operational system, *modulo the evaluator-level transposition*: the re-proof, for a concrete NbE evaluator of that fragment, of the (B)-marked lemmas of Appendices C–D against the normal forms of [5]. The transposition is standard (it is the routine half of every NbE-correctness proof) and involves

> no cubical subtlety beyond what [5] already resolved, but it is a proof obligation, not a citation; and universes, the wider HIT roster, and the De Morgan instance remain outside it.

We prefer this honest statement to the stronger "unconditional" phrasing: the alternative — claiming the evaluator-level lemmas from a theorem that is deliberately not evaluation-based — would misattribute the one part of the base component that their method, precisely by its elegance, does not supply.

References

[1] C. Cohen, T. Coquand, S. Huber, A. Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. *TYPES 2015*, LIPIcs 69, 2018.

[2] C. Angiuli, G. Brunerie, T. Coquand, R. Harper, K.-B. Hou, D. Licata. Syntax and models of Cartesian cubical type theory. *Math. Struct. Comput. Sci.*, 31(4), 2021.

[3] A. Vezzosi, A. Mörtberg, A. Abel. Cubical Agda: a dependently typed programming language with univalence and higher inductive types. *J. Funct. Program.*, 31, 2021.

[4] S. Huber. Canonicity for cubical type theory. *J. Autom. Reasoning*, 63, 2019.

[5] J. Sterling, C. Angiuli. Normalization for cubical type theory. *LICS 2021.*

[6] A. W. Swan. Separating path and identity types in presheaf models of univalent type theory. Preprint, arXiv:1808.00920, 2018.

[7] C. Sattler. The failure of regularity for composition in the CCHM universe (unpublished observation, commonly attributed; we know of no self-contained publication). Published accounts: the introduction and historical remarks of [6], and the discussion of regular composition in [1, 2].

[8] V. Isaev, F. Part, S. Sinchuk. Arend: a theorem prover based on homotopy type theory. JetBrains Research. Language paper and the Prelude documentation of `coe` (whose computation rules include the constant-family collapse) at `https://arend-lang.github.io/documentation/language-reference/prelude`; accessed 2026-07-17.

[9] Y. J. Tan. Towards computational UIP in Cubical Agda. Preprint, arXiv:2511.21209, 2025.

[10] X. Huang. Normal forms in cubical type theory. Preprint, arXiv:2603.24923, 2026.

[11] I. Orton, A. M. Pitts. Axioms for modelling cubical type theory in a topos. *LMCS* 14(4), 2018.

[12] J. Sterling, C. Angiuli, D. Gratzer. Cubical syntax for reflection-free extensional equality. *FSCD 2019.*